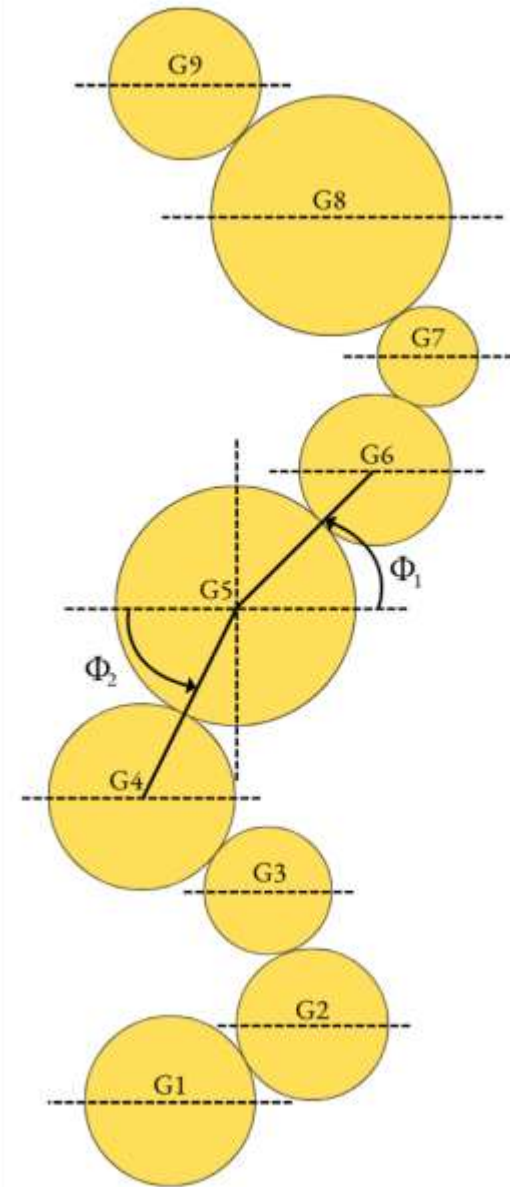


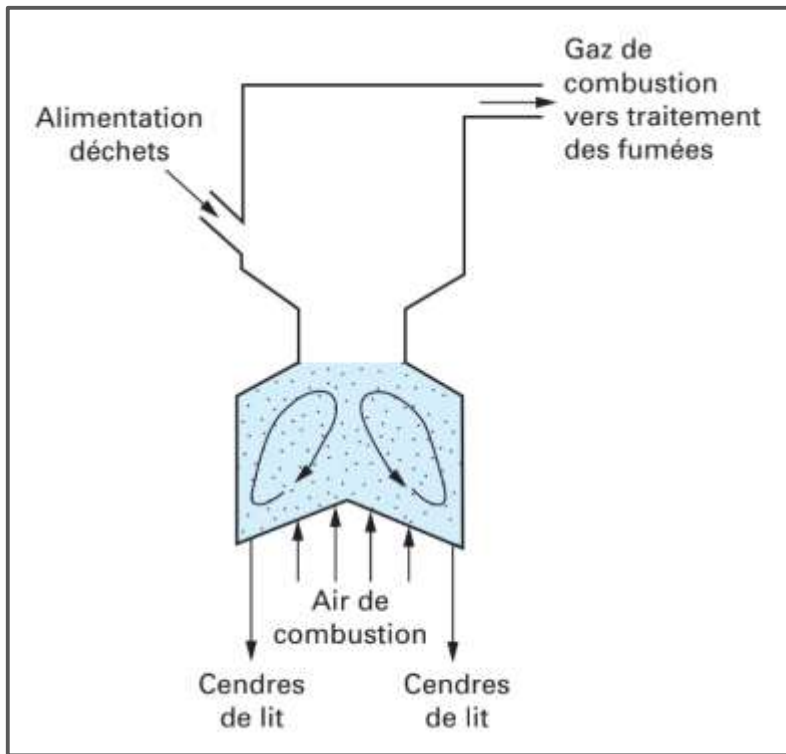
Fluidisation des milieux granulaires: principe et applications

Thème 2025: Transformation, transition, conversion

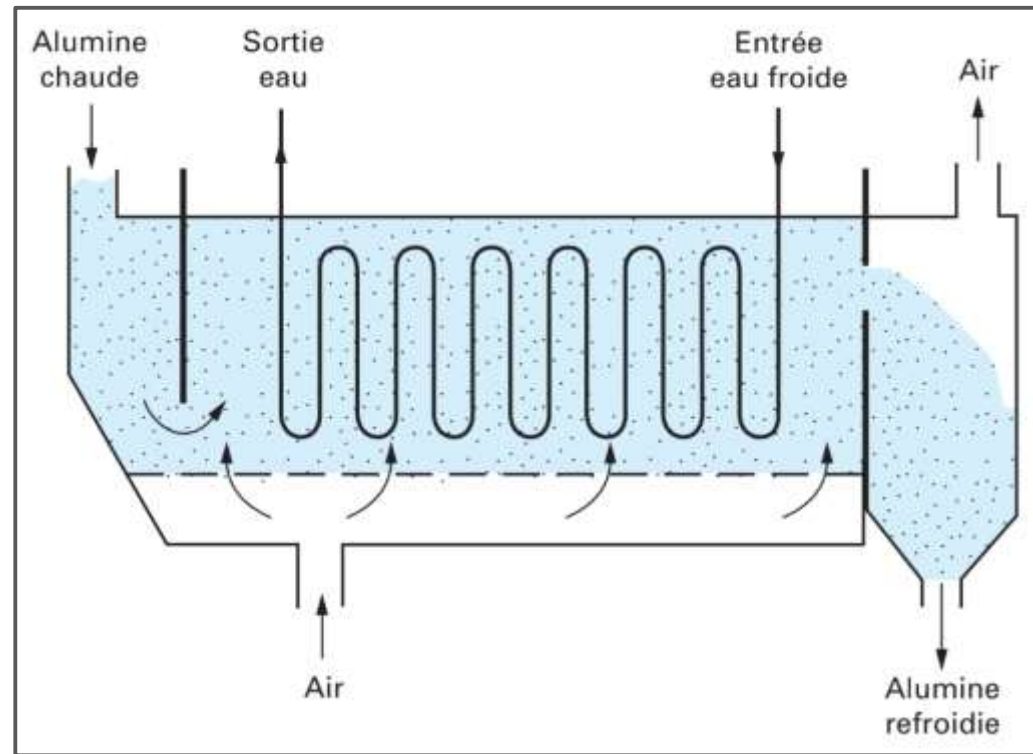
BAUDET Even 48762



Introduction

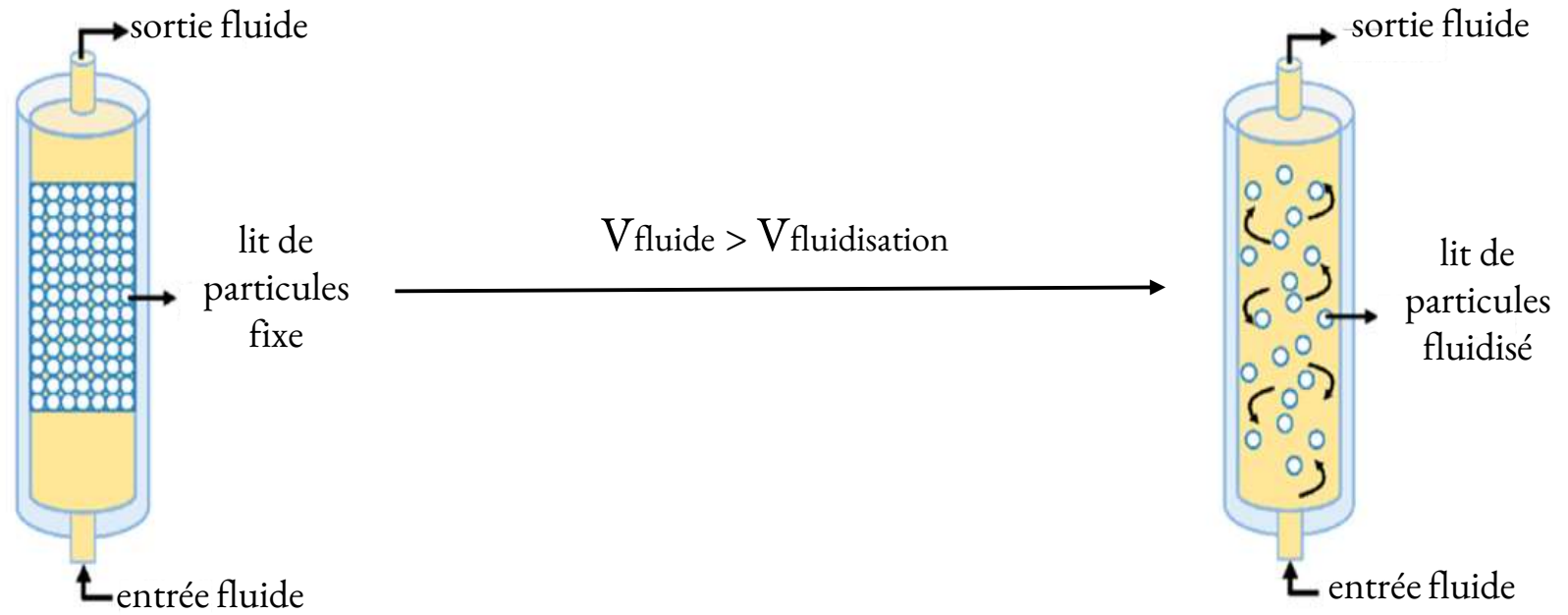


four d'incinération à lit fluidisé



système de refroidissement d'alumine en lit fluidisé

Principe



Objectifs du TIPE

- Observer le phénomène de fluidisation
- Validation des lois régissant la fluidisation
- Etude de différents paramètres

objectif personnel: réaliser un modèle numérique permettant de rendre compte du phénomène de fluidisation

I - Mise en place du montage et des outils de mesure

II - Vérification des lois de Darcy et d'Ergun

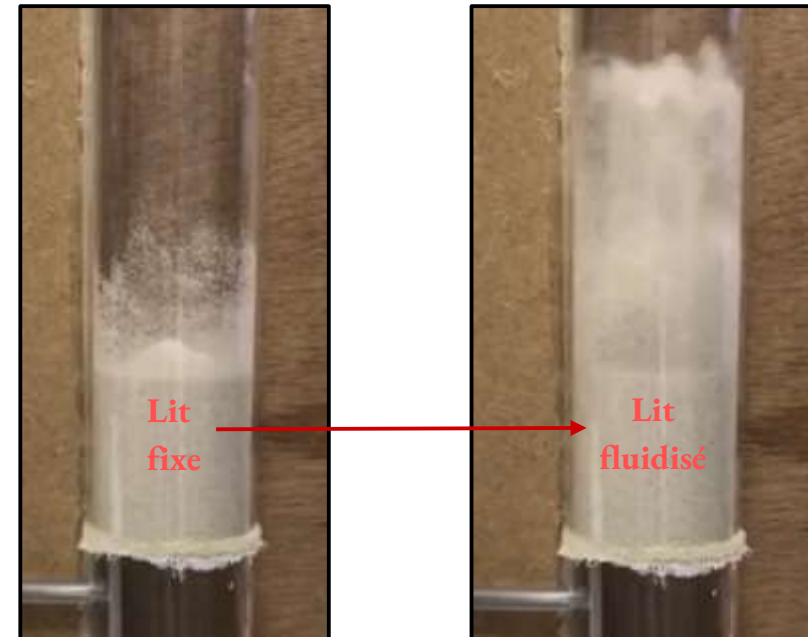
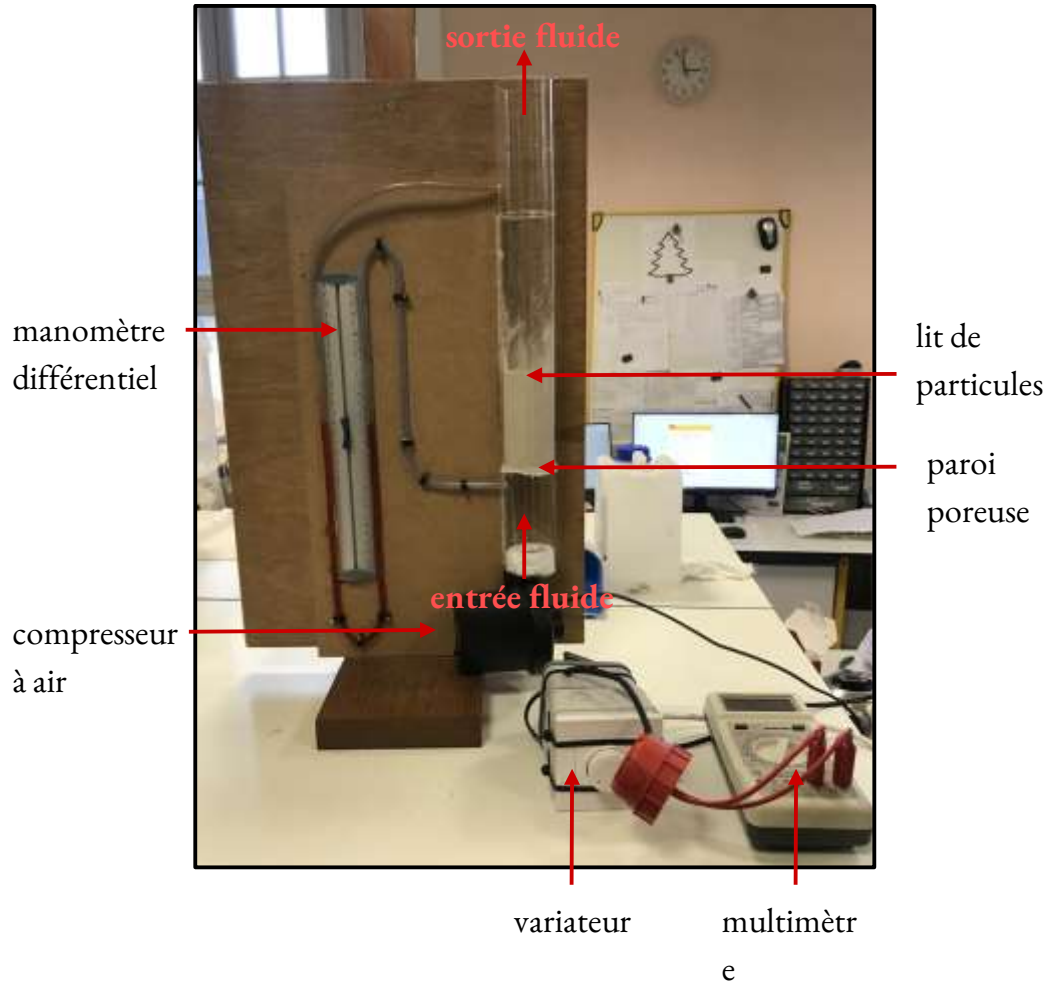
III - Développement d'un modèle numérique

IV - Validation du modèle

I - Mise en place du montage

Premier montage

photo légendée du montage



observation du phénomène de fluidisation

I - Mise en place du montage

Problèmes rencontrés

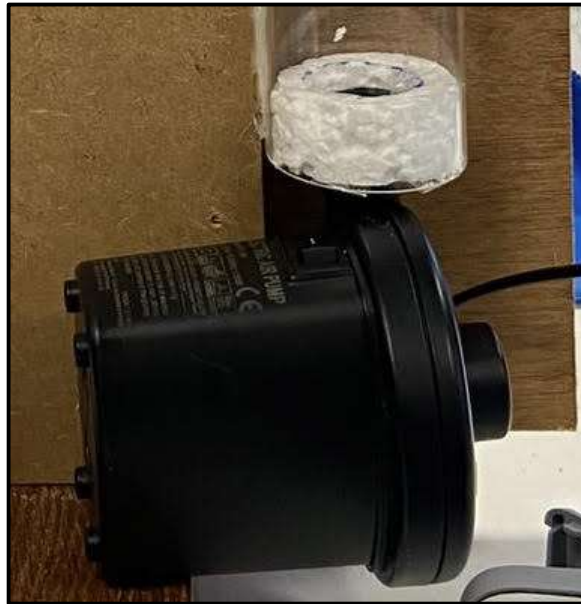
Manomètre différentiel

- incertitude élevée
- mesures limitées ($\Delta h < 30\text{cm}$)



**1 → changement d'outil
de mesure de pression**

Compresseur à air



**2 → détermination de la
caractéristique débit-tension**

Paroi poreuse



**3 → mesure de la composante de
surpression liée à la paroi**

I - Mise en place du montage

1 - Changement d'outil de mesure de pression

Montage Final



manomètre différentiel débranché

mesures effectuées dans une cocotte minute

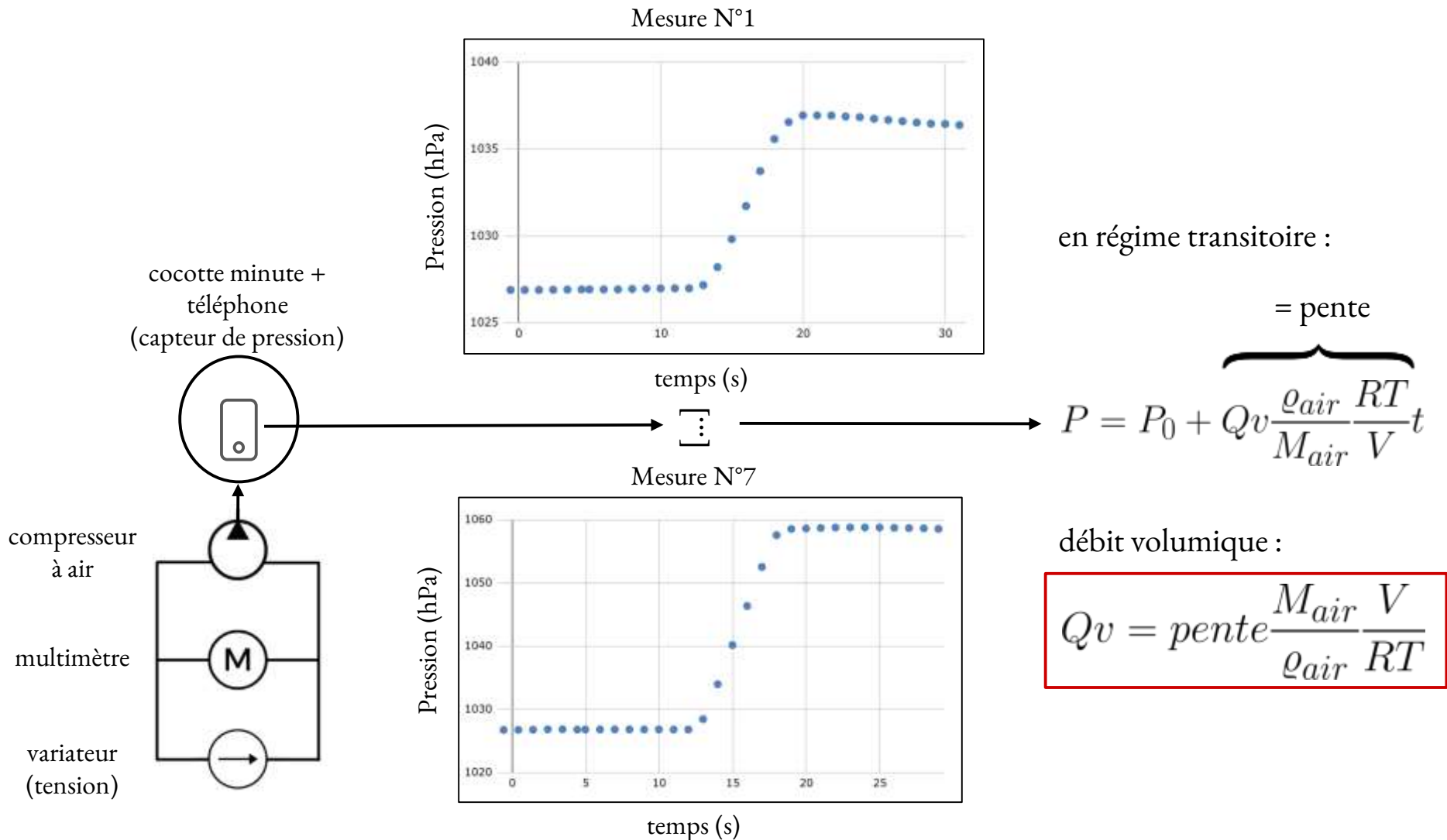
Avantages:

- mesures réalisées avec l'application phyphox (précision de l'ordre du pascal)
- mesures non limitées aux faibles pression



I - Mise en place des outils de mesure

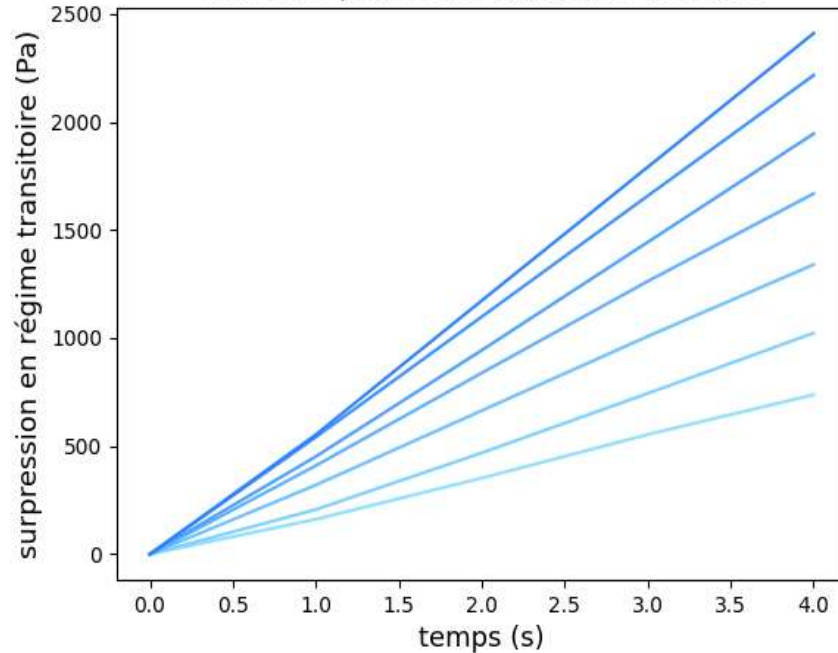
1 - Mesure de la caractéristique débit - tension : protocole



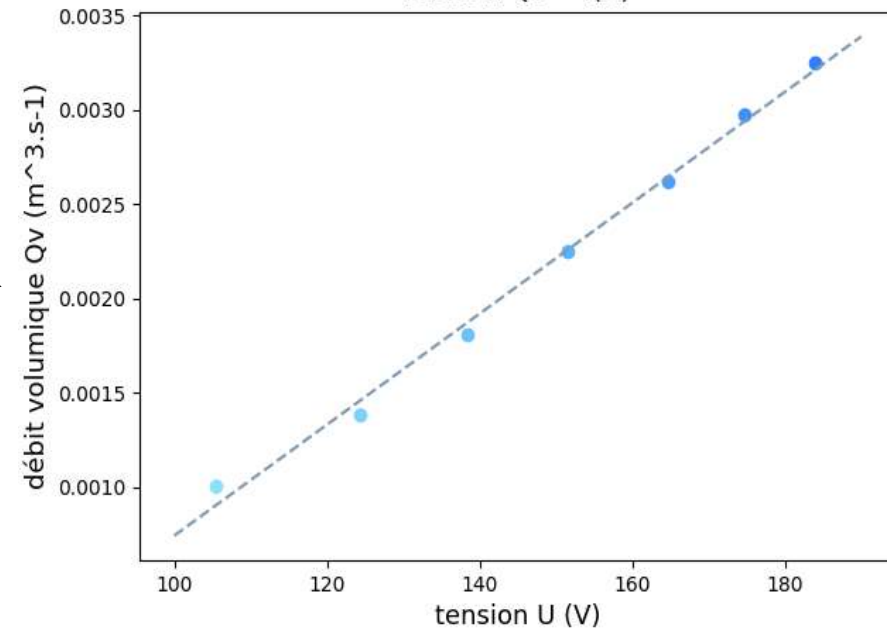
I - Mise en place des outils de mesure

1 - Mesure de la caractéristique débit - tension : résultats

tracé des pentes des différentes mesures



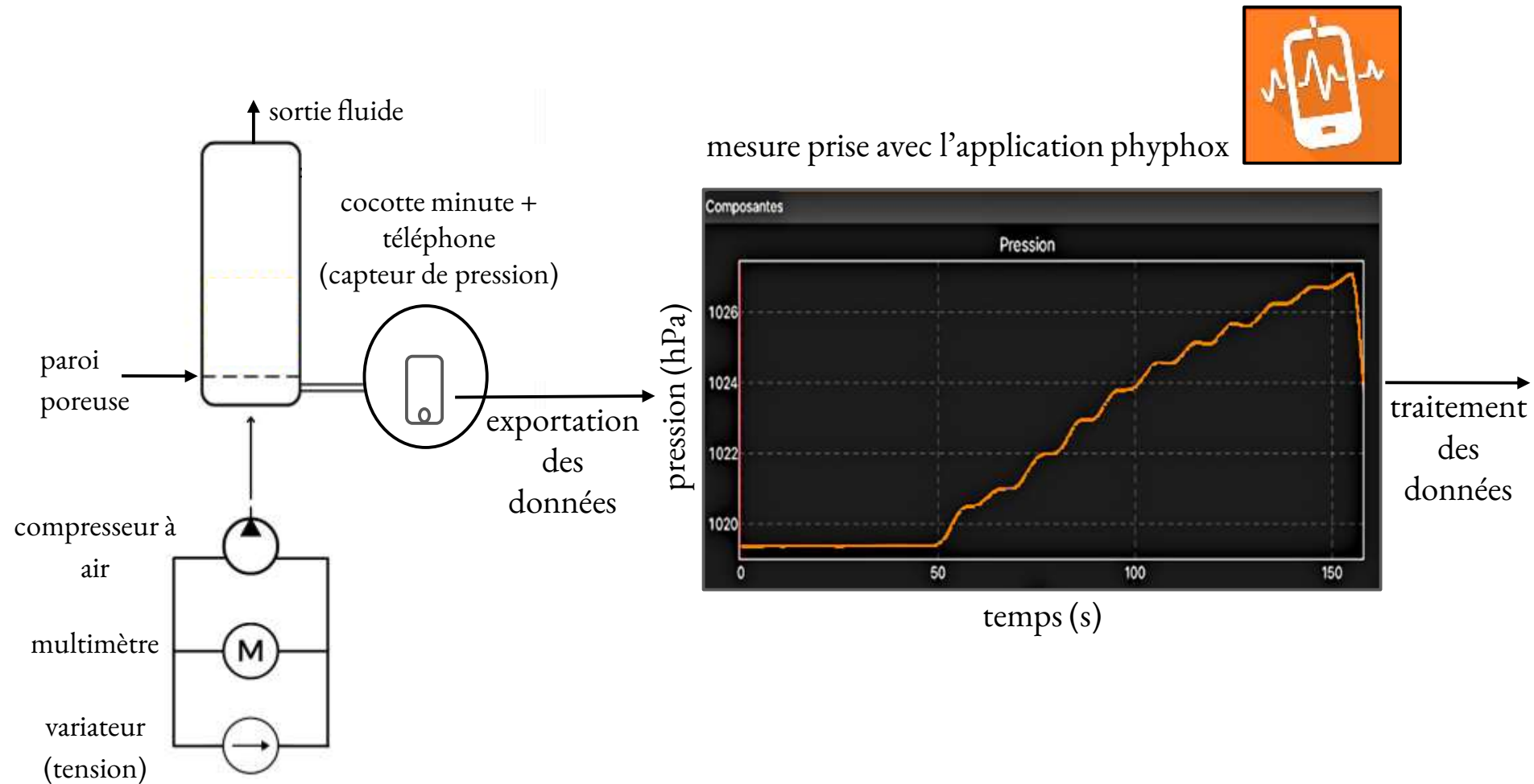
relation $Q_v = f(U)$



caractéristique linéaire sur les valeurs de débit d'intérêt

I - Mise en place des outils de mesure

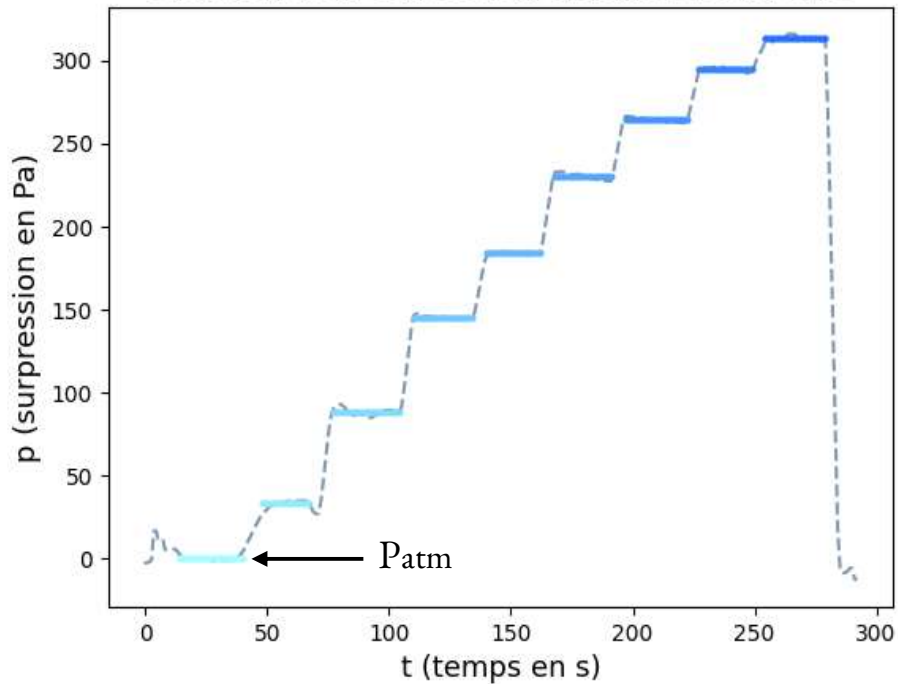
1 - Mesure de la composante liée à la paroi



I - Mise en place des outils de mesure

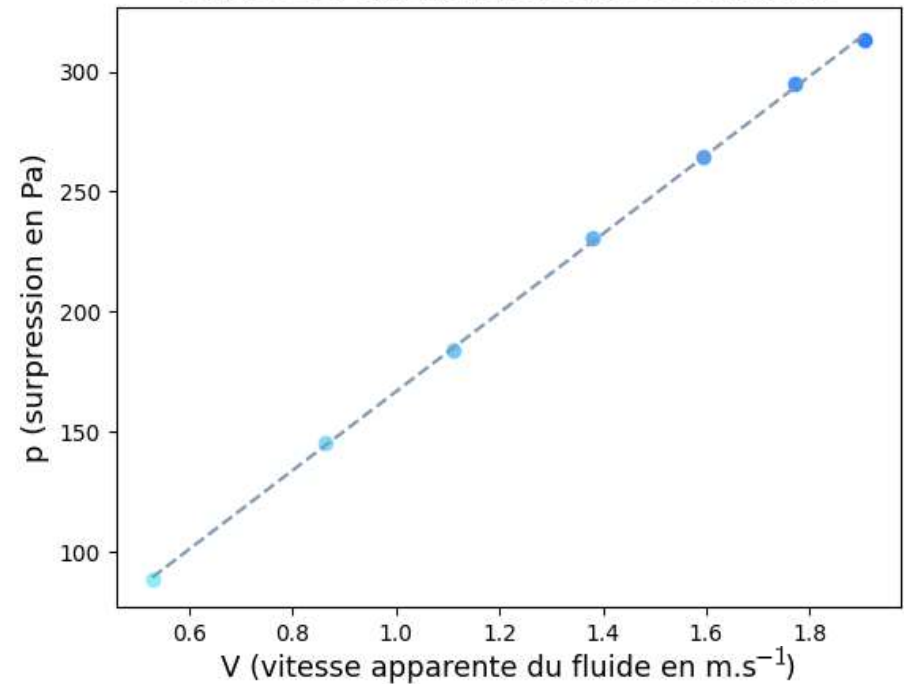
1 - Mesure de la composante liée à la paroi

évolution de la surpression en fonction du temps



paliers déterminés avec un script python

surpression en fonction de la vitesse du fluide



composante de surpression
linéaire et non négligeable

Plan de l'étude

I - Mise en place du montage et des outils de mesure

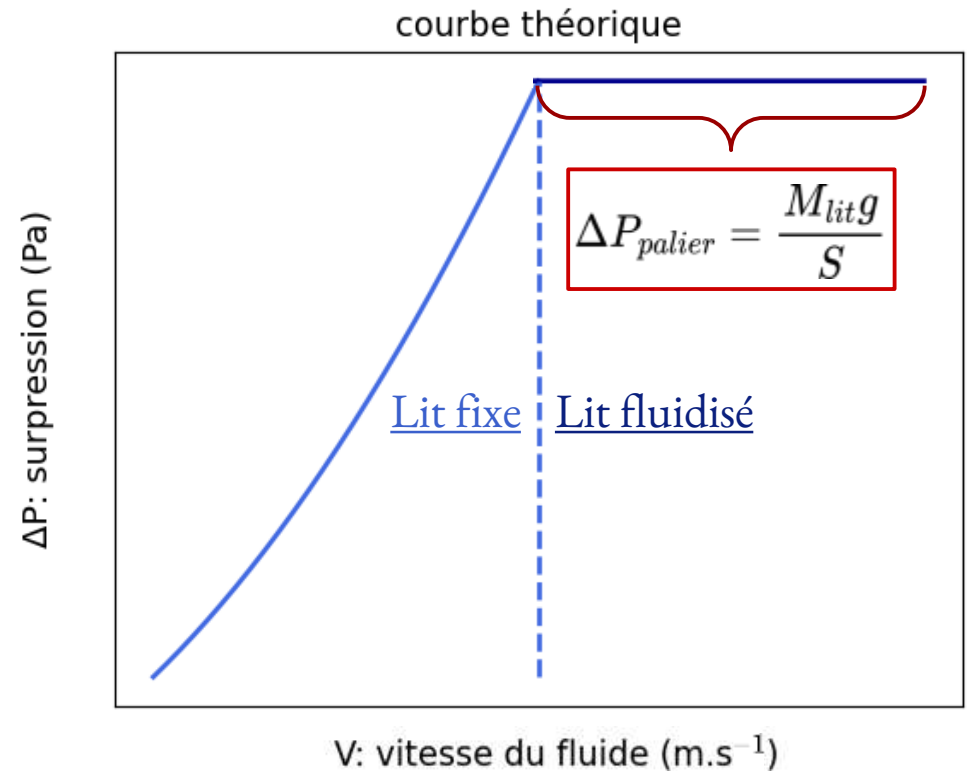
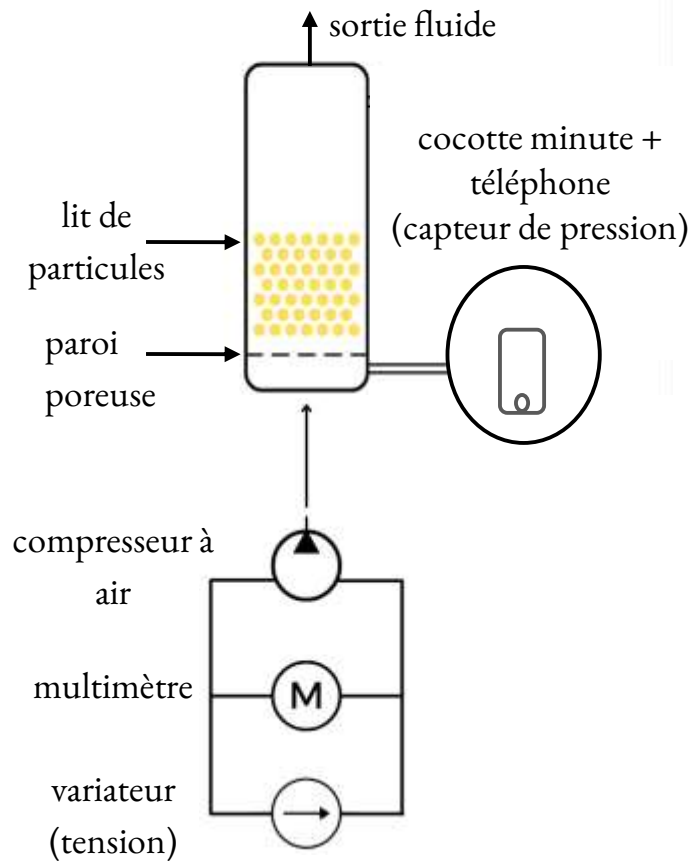
II - Vérification des lois de Darcy et d'Ergun

III - Développement d'un modèle numérique

IV - Validation du modèle

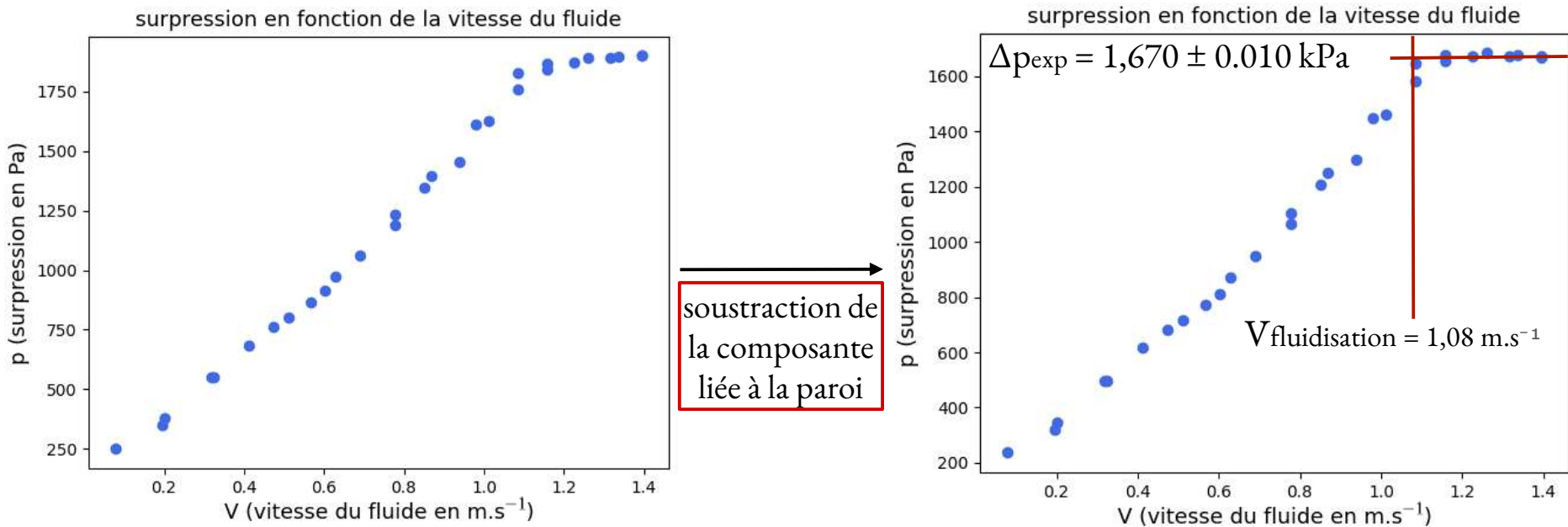
II - Vérification des lois régissant le phénomène de fluidisation

1 - Palier de surpression après le seuil de fluidisation



II - Vérification des lois régissant le phénomène de fluidisation

1 - Résultats obtenu



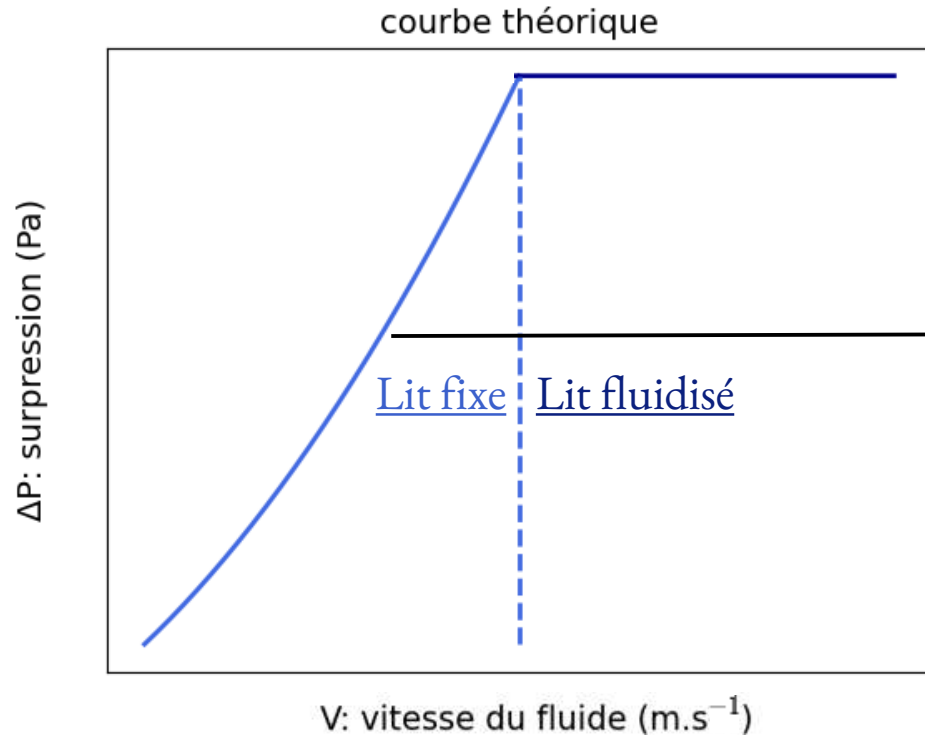
palier de surpression après fluidisation

$$\Delta p_{\text{th}} = 1,651 \text{ kPa}$$

$$\varepsilon_n = 1,7 < 2$$

II - Vérification des lois régissant le phénomène de fluidisation

1 - Loi de Darcy et loi semi-empirique d'Ergun



équation en lit fixe:

Loi de Darcy

$$\frac{\Delta p}{L} = \frac{\eta}{k} V$$

Loi semi-empirique d'Ergun

$$\frac{\Delta p}{L} = \frac{\eta}{k} V + 1,8 \frac{\rho_{\text{fluide}}}{d_p} \frac{\phi}{(1 - \phi)^3} V^2$$

$\phi = \frac{V_{\text{grains}}}{V_{\text{total}}}$: fraction volumique

d_p : diamètre des particules (m)

k : perméabilité du milieu (m^2)

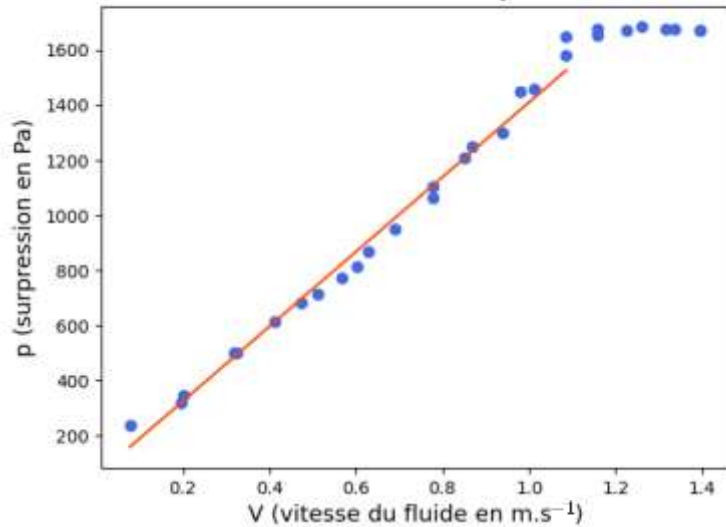
η : viscosité dynamique ($\text{kg.m}^{-1}.\text{s}^{-1}$)

L : hauteur du lit (m)

II - Vérification des lois régissant le phénomène de fluidisation

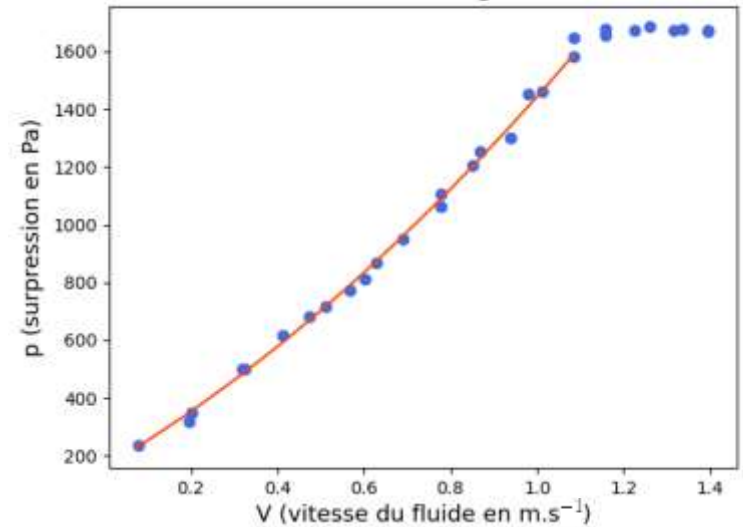
1 - Tracé des deux modèles (Darcy/Ergün)

modèle de Darcy

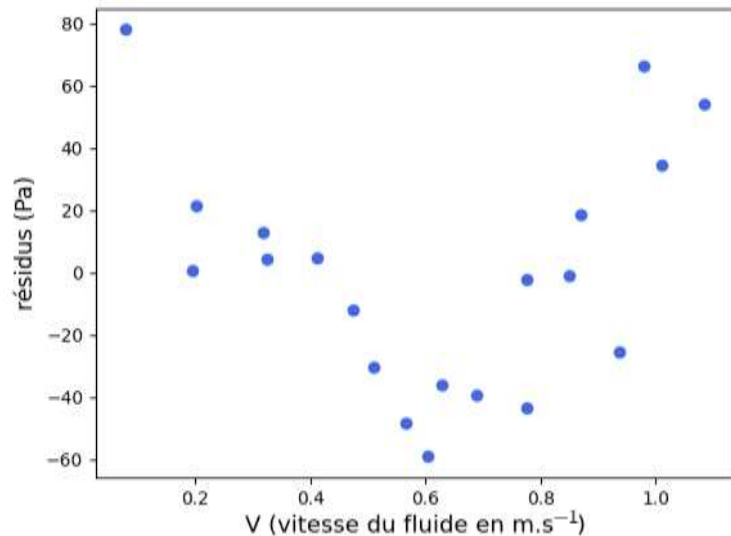


mesures effectuées

modèle de Ergün

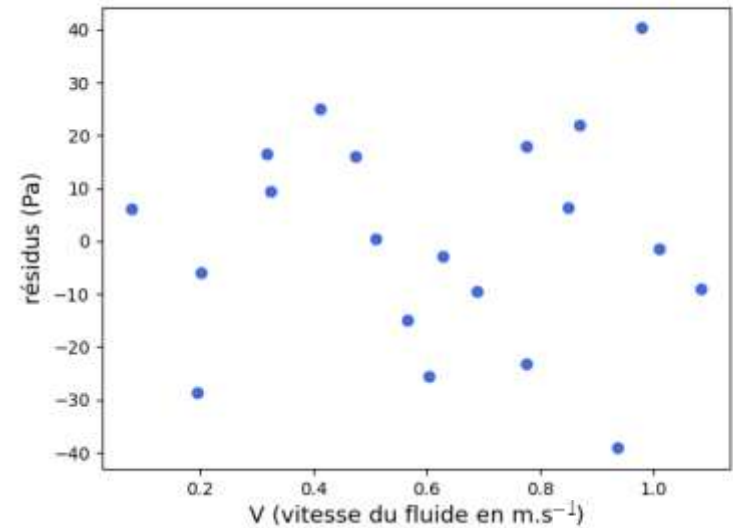


tracé des résidus



tracé des résidus

tracé des résidus



Plan de l'étude

I - Mise en place du montage et des outils de mesure

II - Vérification des lois de Darcy et d'Ergun

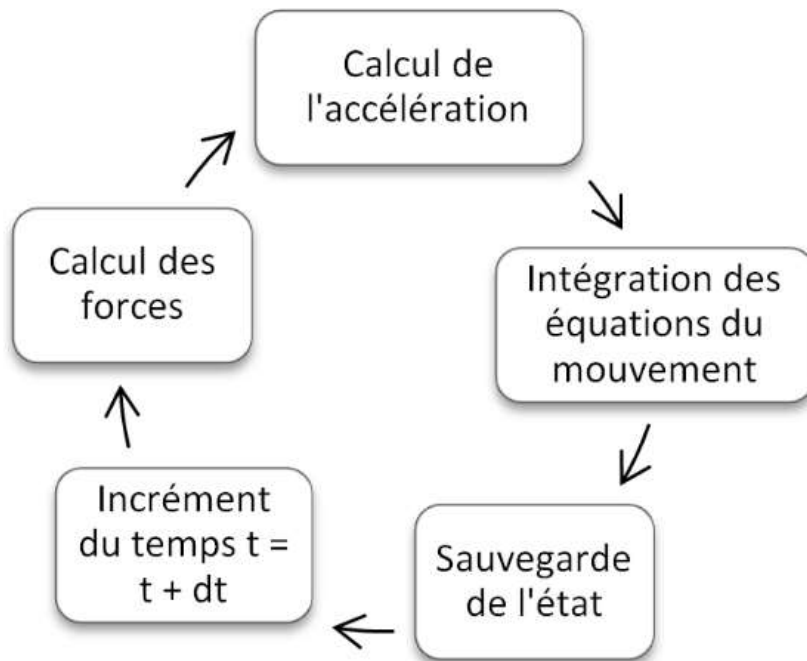
III - Développement d'un modèle numérique

IV - Validation du modèle

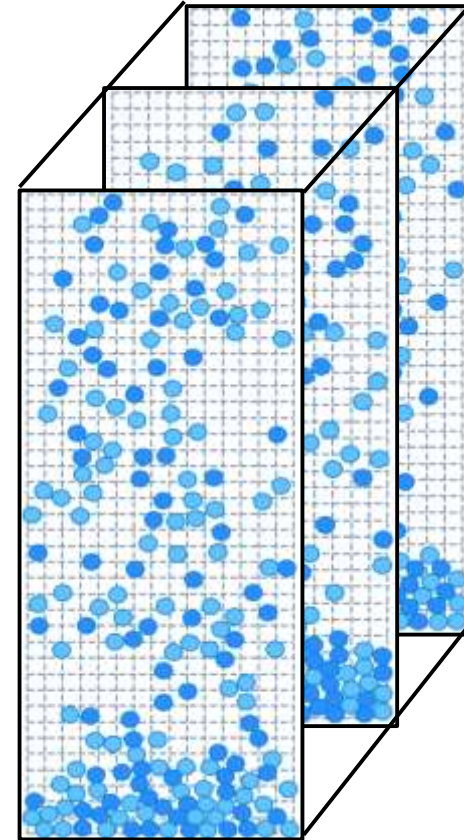
III - Développement d'un modèle numérique

1 - Méthode des éléments discrets

Principe



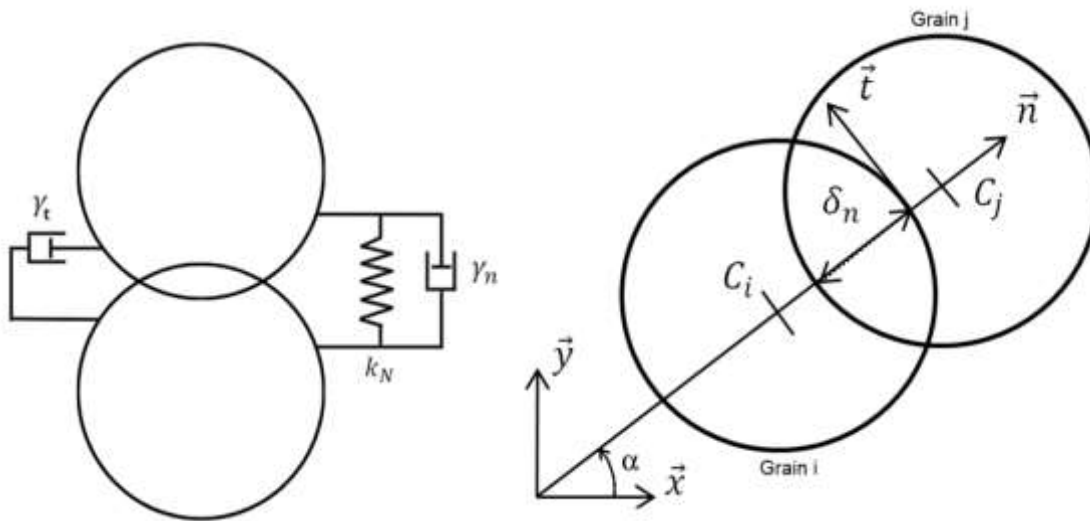
Etude d'un plan de coupe du montage



III - Développement d'un modèle numérique

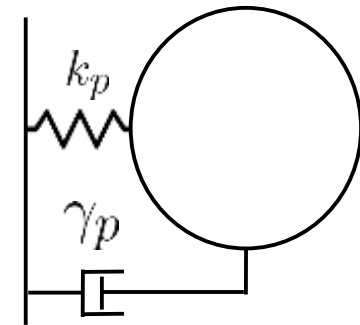
2 - Modélisation des forces de collisions

Interaction entre grains : variante de la méthode des sphères molles



Les Milieux Granulaires: entre fluide et solide (CNRS éditions, 2011)

Interaction des grains avec la paroi (ressort et frottement) :

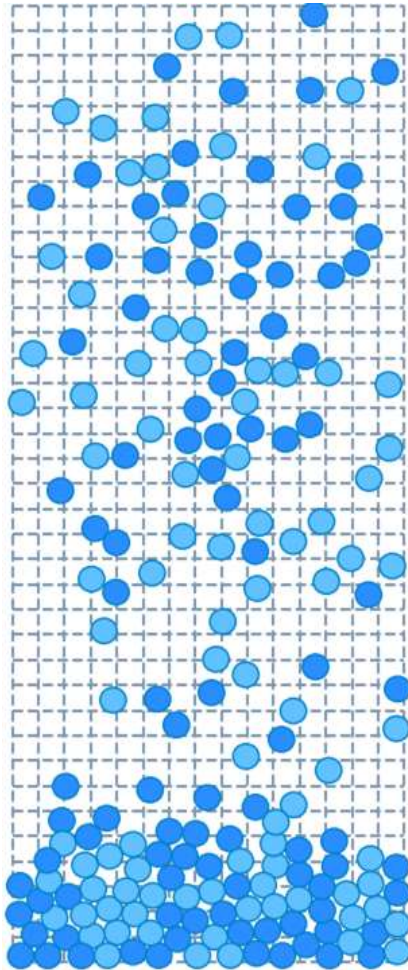


condition aux limites: inversion de la composante normale de la vitesse au contact de la paroi

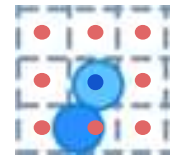
III - Développement d'un modèle numérique

2 - Détection des collisions

1 - Algorithme de répartition des grains dans un tableau en $O(N)$



2 - Détermination des interactions limité aux plus proche voisins



→ étude d'au plus 9 cases

complexité totale de l'algorithme en $O(N)$
(N : nombre de grains)

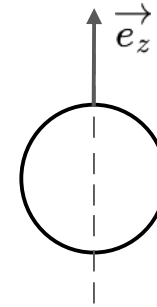
→ efficace pour des simulations au
niveau industriel

III - Développement d'un modèle numérique

2 - Modélisation des forces liées au fluide

Interaction grains/fluide : force de pression appliquée sur chaque grain

$$\overrightarrow{F_p}(V, \phi) = \frac{4\pi}{3} r^3 \frac{\partial P}{\partial z}(V, \phi) \overrightarrow{e_z}$$



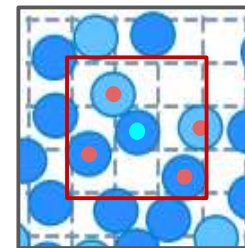
Calcul du gradient de pression
donné par la loi d'Ergün :

$$\frac{\partial P}{\partial z}(V, \phi) = A(\phi)V^2 + B(\phi)V$$

$$\text{avec } A(\phi) = \frac{\rho_{air}}{d_p} \frac{\phi}{(1 - \phi)^3}$$

$$\text{et } B(\phi) = \frac{150}{d_p^2} \frac{\phi^2}{(1 - \phi)^3} \eta$$

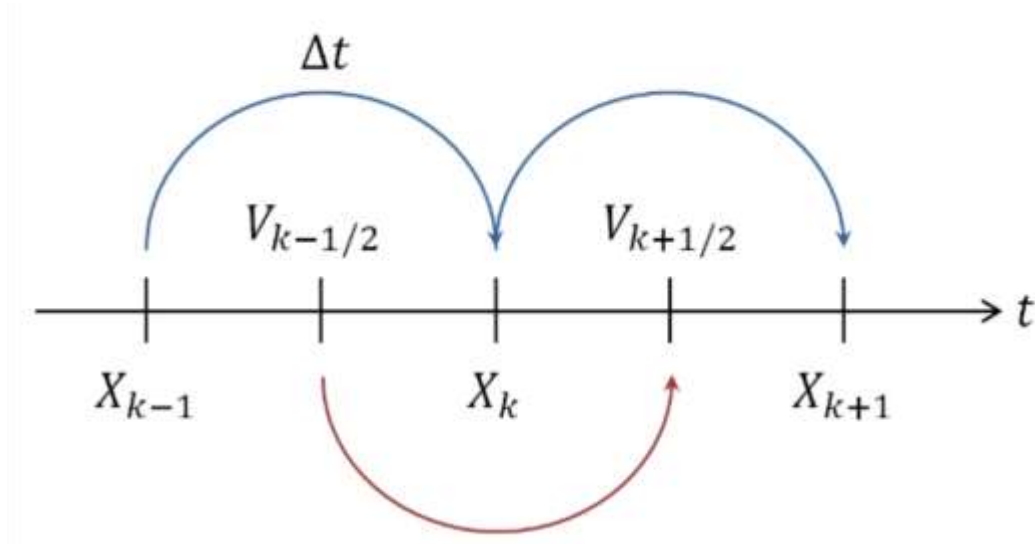
Calcul de la fraction volumique :



$$\phi = \frac{V_{grains}}{V_{total}}$$

III - Développement d'un modèle numérique

3 - Méthode d'intégration choisie - Méthode de Verlet (Leapfrog)



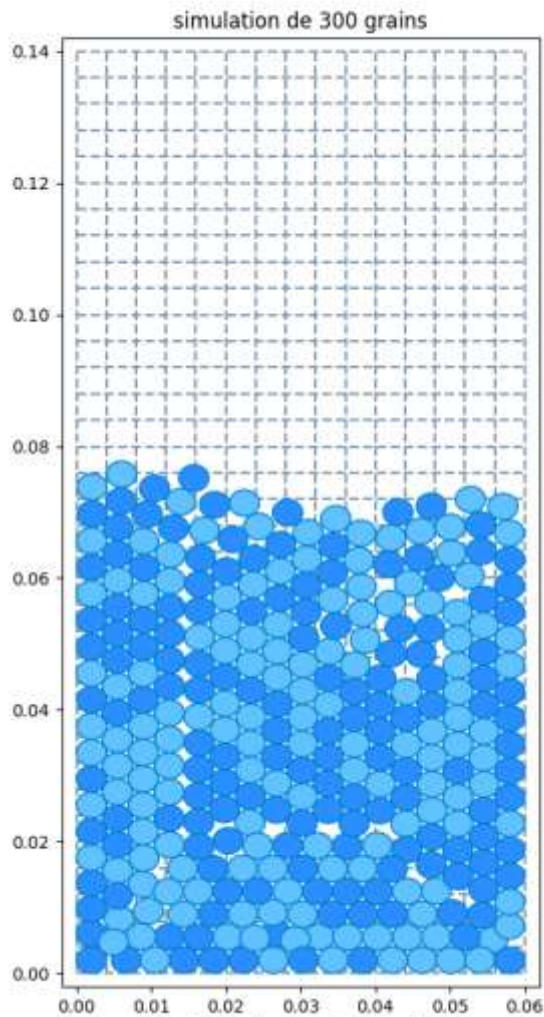
$$X(t - \Delta t) = X(t) - \underbrace{v(t)\Delta t + \frac{1}{2}a(t)\Delta t^2}_{= -V_{k-1/2}}$$

$$X(t + \Delta t) = X(t) + \underbrace{v(t)\Delta t + \frac{1}{2}a(t)\Delta t^2}_{= V_{k+1/2}}$$

$$\text{donc } X(t + \Delta t) = 2X(t) - X(t - \Delta t) + a(t)\Delta t^2 + \boxed{\mathcal{O}(\Delta t^4)}$$

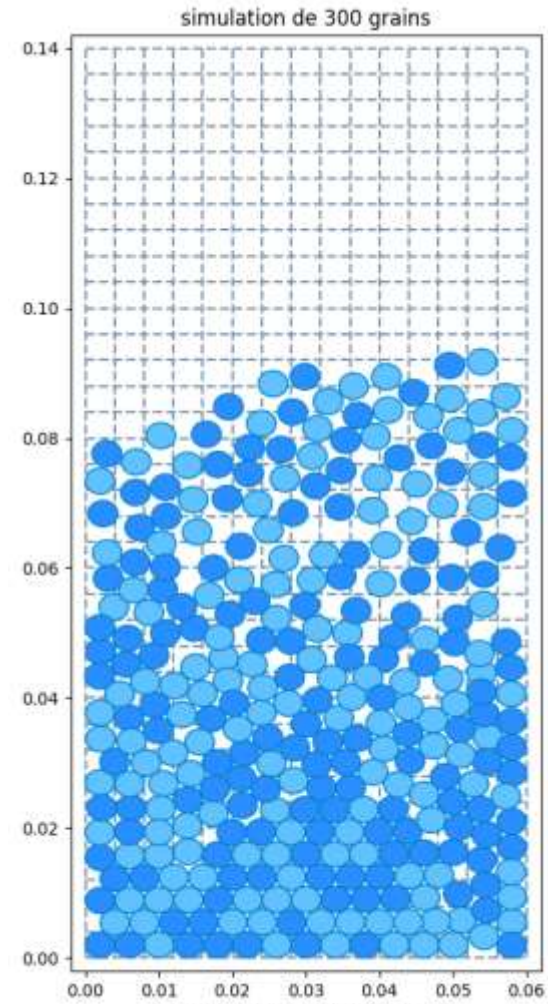
III - Développement d'un modèle numérique

4 - Premiers résultats



abscisse x et ordonnée y en mètres

$\xrightarrow{V_{\text{fluide}} > 1,5 \text{ m.s}^{-1}}$



abscisse x et ordonnée y en mètres

Plan de l'étude

I - Mise en place du montage et des outils de mesure

II - Vérification des lois de Darcy et d'Ergun

III - Développement d'un modèle numérique

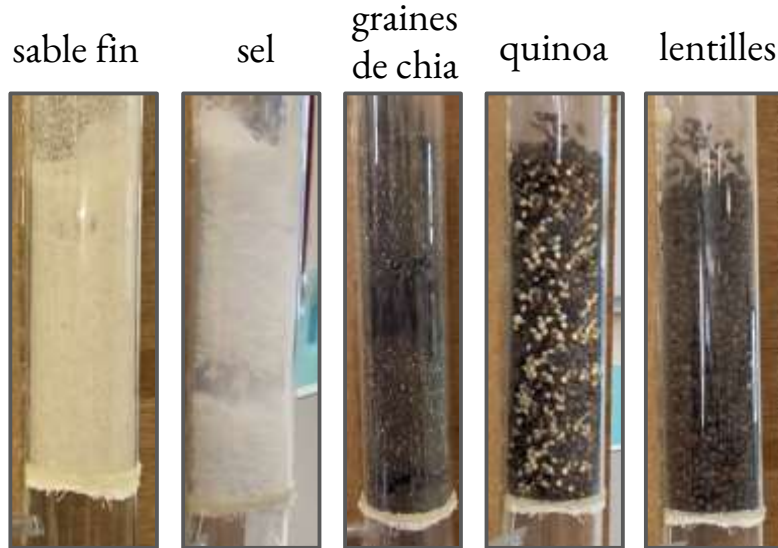
IV - Validation du modèle

IV - Validation du modèle

1 - Détermination des vitesses de fluidisation pour différent milieux

Détermination expérimentale

milieux granulaires à disposition :



1 → détermination des caractéristiques de chaque milieu:

- masse volumique des grains
- rayon des grains

2 → détermination des vitesses de fluidisation:

Détermination numérique

```
#variables concernant un grain  
rp = 6e-4 #rayon des grains en m  
N = 300 #nombre de grains  
m = 2.4e-6 #masse d'un grain (kg)
```

simulation:

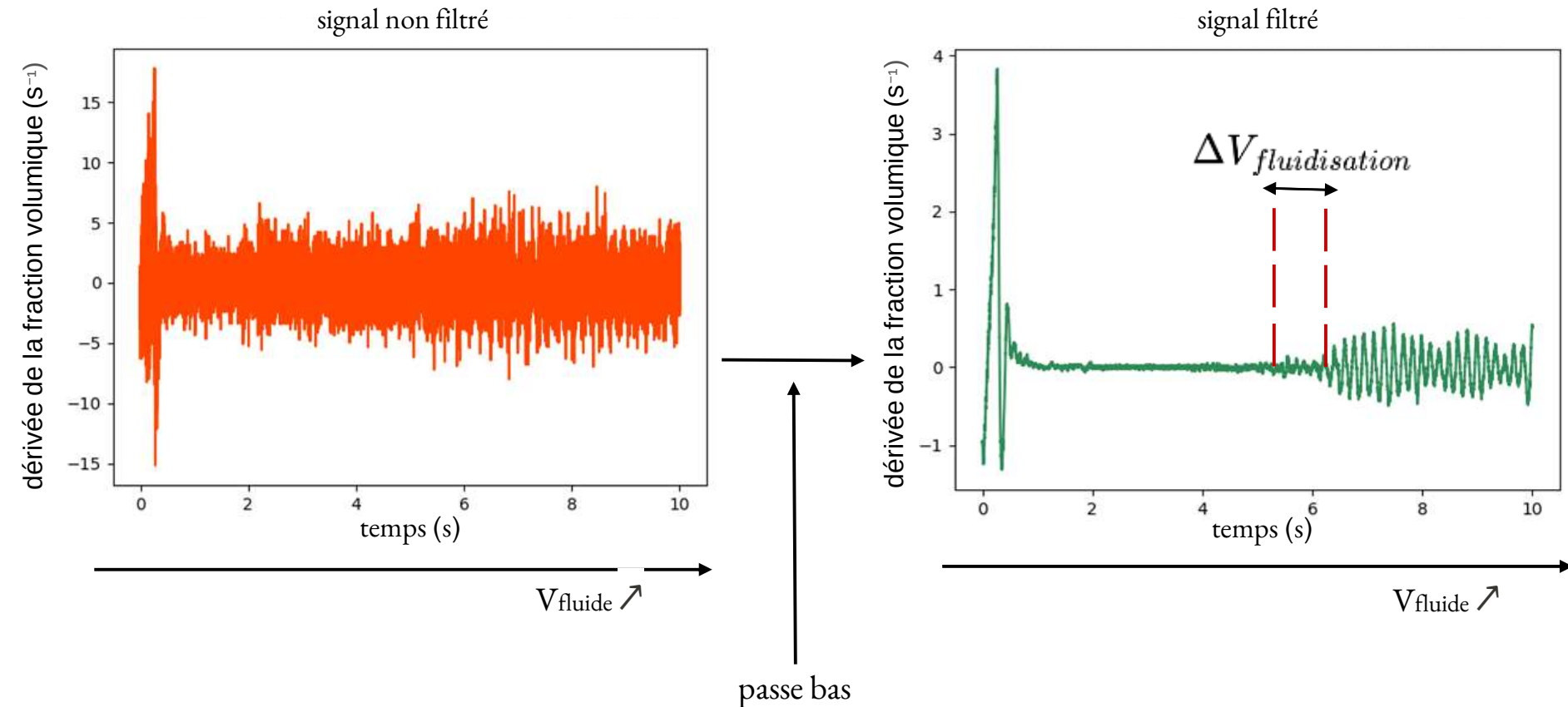
- V augmente linéairement avec le temps
- On enregistre les valeurs de ϕ et de V

```
#nouvelles positions, vitesses, accelerations au temps t + dt  
Ax,Ay,Vx,Vy,X,Y = integration(X,Y,Vx,Vy,Ax,Ay)  
  
t += dt  
V += dV  
  
phi_M = np.mean(phi)  
  
L_phi.append(phi_M)  
L_V.append(V)
```

Tracé de la dérivée de ϕ en fonction du temps

IV - Validation du modèle

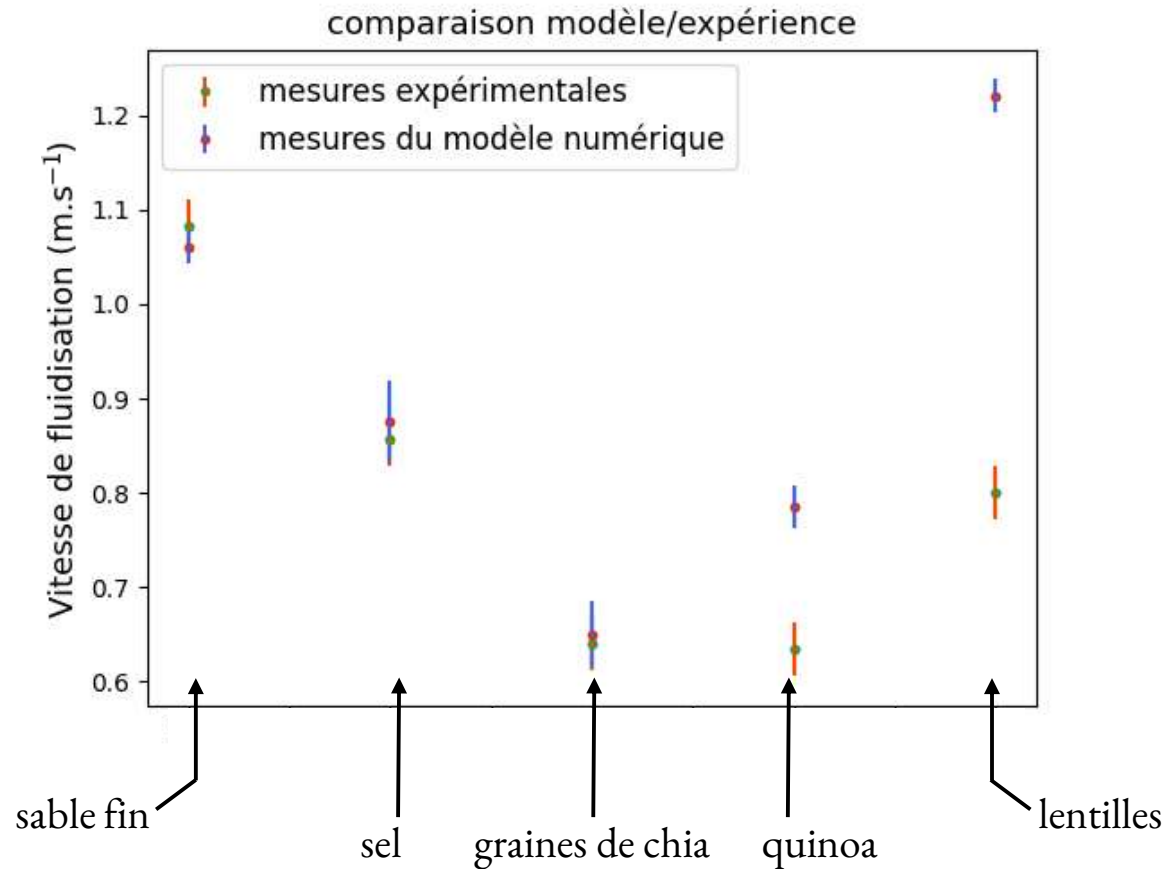
1 - Détermination des vitesses de fluidisation pour différent milieux



```
#filtrage des hautes frequences
tau = 2.5e-2
d_phi_filtre = [d_phi[0]]
for i in range(1,len(d_phi)):
    d_phi_filtre.append(d_phi_filtre[i-1] + (dt/tau)*(d_phi[i-1] - d_phi_filtre[i-1]))
```

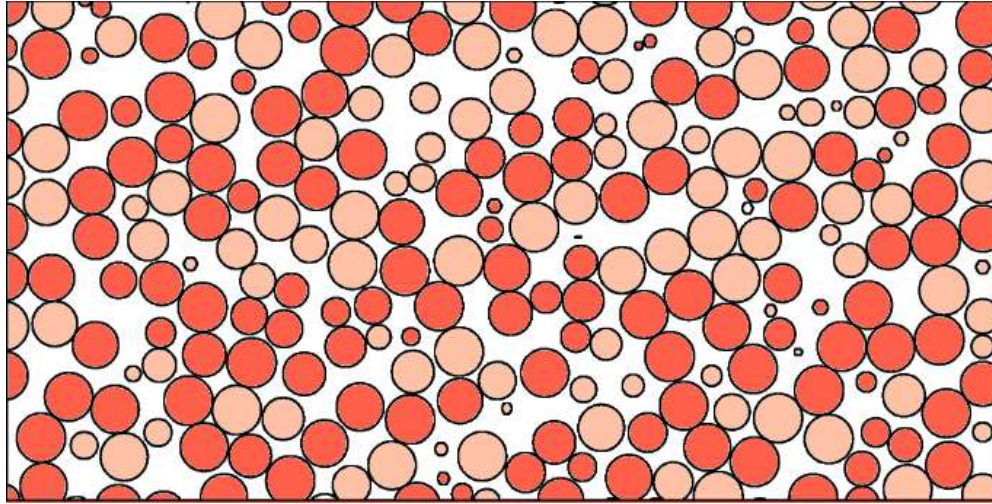
IV - Validation du modèle

2 - Comparaison des résultats : modèle numérique/expérience

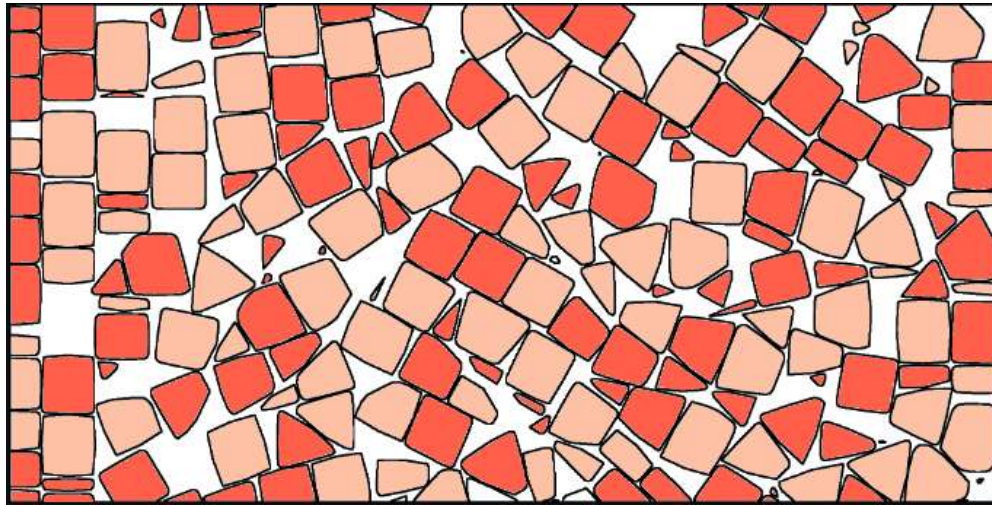


Conclusion

Effets non pris en compte



Mélange de grains de taille différente:
→ empilement compact :
augmentation de la fraction volumique



Milieus granulaires non sphériques :
→ variation des lignes de champ
du fluide au sein de l'empilement

Merci de votre attention

Annexe

1 - Tracé de courbes : relation $U = f(Q_v)$

```
1  ### importation des modules
2  import matplotlib.pyplot as plt
3  import matplotlib.colors as pltc
4  import numpy as np
5
6  ### liste des mesures de temps
7  t = [[13.97213008,14.97269608,15.97328108,16.97385008,17.97442908],
8        [16.97438462,17.97492362,18.97550662,19.97607862,20.97665662],
9        [15.97356608,16.97414208,17.97472308,18.97529708,19.97587108],
10       [14.99009146,15.99066946, 16.99124446,17.99182546,18.99239646],
11       [10.98806779,11.98864479,12.98921779,13.98979179,14.99037479],
12       [12.97038087,13.97110888,14.97154287,15.97211588, 16.97269088],
13       [12.97444912,13.97502613,14.97559912,15.97617512, 16.97674712]]
14
15  ### liste des mesures de pression
16  p = [[1028.202667,1029.816666,1031.722565,1033.73558, 1035.580292],
17        [1027.654266,1029.717178,1032.341156, 1035.089874,1037.885818],
18        [1029.192047, 1032.389755,1035.835266,1039.269333,1042.591248],
19        [1029.902573,1033.9991, 1038.287277,1042.528458,1046.587601],
20        [1028.828354,1033.344574,1038.263397,1043.262558,1048.282623],
21        [1030.326462,1035.741653,1041.311798, 1046.902618,1052.486191],
22        [1028.475723,1034.004517,1040.20874, 1046.385269,1052.576599]]
23
24
25  ### initialisation des listes
26  a = [] #liste des pentes
27  couleur = [] #liste des couleurs des pentes
28
29  ### création de la figure 1
30  n = len(p)
31  plt.figure()
32  for i in range(n):
33      c=pltc.to_hex(((140 - 16*i)/255, (225 - 16*i)/255,1)) #différenciation des pentes par couleur
34      temps, pression = np.array(t[i]), np.array(p[i])*1e2 #conversion en Pa
35      plt.plot(temps - temps[0], pression - pression[0], color = c)
36      a.append(np.polyfit(temps, pression,1)[0])
37      couleur.append(c)
38
39  plt.xlabel('temps (s)', fontsize = 13)
40  plt.ylabel('surpression en régime transitoire (Pa)', fontsize = 13)
41  plt.title('tracé des pentes des différentes mesures', fontsize = 13)
42  plt.show()
```

Annexe

1 - Tracé de courbes : relation $U = f(Q_v)$

```
44  ### calcul et tracé des débits volumiques
45  U = np.array([105.5,124.25,138.4,151.5,164.75,174.7,183.9])
46  V = 5.5e-3
47  T = 298
48  R = 8.314
49  rho = 1.2
50  M = 29e-3
51  pente = np.array(a)
52  Qv = pente*1e2*M*V/(R*T*rho)
53
54  plt.figure()
55  for i in range(n):
56      plt.plot(U[i], Qv[i], 'o', color = couleur[i])
57
58  ### droite modèle
59  a,b = np.polyfit(U,Qv,1)
60  m_U = np.array([5*k for k in range(20,39)])
61  m_Qv = a*m_U + b
62  print(a,-b/a)
63
64  ### tracé de la figure
65  plt.plot(m_U,m_Qv, '--', color = pltc.to_hex((0.5,0.6,0.7)))
66  plt.xlabel('tension U (V)', fontsize = 13)
67  plt.ylabel('débit volumique Qv (m3.s-1)', fontsize = 13)
68  plt.title('relation Qv = f(U)', fontsize = 13)
69  plt.show()
```

Annexe

1 - Algorithme de recherche des paliers de surpression

```
1  %%% importation des modules
2  import matplotlib.pyplot as plt
3  import numpy as np
4
5  %%% algorithme de recherche des paliers de surpression (d_p: dérivée de la pression, p: pression, t:temps, K:constante)
6  def intervalles(d_p,p,t,K):
7      #T,P listes qui vont contenir les valeurs de temps et de pression aux paliers
8      #P_f: liste des moyennes des valeurs de surpression aux paliers
9      P,T,P_f = [],[],[]
10
11
12      %%%parcours de toutes les valeurs
13      N = len(d_p)
14      i = 0
15      while i < N:
16
17          %%% conditions pour que les valeurs soient considérées comme un palier
18          #si la dérivée de p est inférieure en valeur absolue à K (K déterminée par essais successifs)
19          #et si au moins les 10 valeurs suivantes vérifient la même condition, alors ces valeurs sont un palier
20          if abs(d_p[i]) < K:
21              j = i
22              while j < N and abs(d_p[j]) < K:
23                  j += 1
24              if j > i+10:
25                  T.append(t[i:j+1])
26                  P.append(p[i:j+1])
27              i = j
28              i += 1
29
30      %%% calcul des moyennes de surpression aux paliers
31      for i in range(len(T)):
32          a = np.mean(P[i])
33          P_f.append(a)
34      return T,P_f
```

Annexe

1 - Tracé de courbes : composante linéaire de surpression due à la paroi

```
59  """ valeurs des paliers de surpression déterminées grâce à l'algorithme de recherche des paliers de surpression
60  p_palier = np.array([ 88.43276205, 145.43715679, 183.97087147, 230.38046969,
61                      264.34228294, 294.74500648, 313.43617188])
62
63  """ calcul des valeurs de vitesses grâce à la caractéristique débit-tension
64  U = np.array([117.5 , 144.5 , 164.5 , 186.25, 203.5 , 217.8 , 228.85])
65
66  #relation  $V = m*(U-U_0)$  avec V la vitesse apparente du fluide
67  m,U0 = 2.9407202207059275e-05, 74.7729332913398
68
69  #Section du tube en m2
70  S = np.pi*((5.5e-2)/2)**2
71  m = m/S
72
73  #relation entre vitesse et tension
74  V = m*(U - U0)
75
76  """ vérification d'une relation linéaire entre p et V
77  (c,d) = np.polyfit(V,p_f,1)
78
79  """
80  plt.figure()
81  for i in range(len(p_f)):
82      plt.plot(V[i],p_f[i],'o', color = couleur[i+1])
83  plt.plot(V,c*V + d,'--', color = '#8099b2')
84  plt.xlabel('V (vitesse apparente du fluide en m.s-1)')
85  plt.ylabel('p (surpression en Pa)')
86  plt.show()
```

Annexe

1 - Tracé de courbes : relation surpression = $f(V_{\text{fluide}})$

```
87  %% valeurs de surpression dont on a soustrait la composante linéaire lié à la paroi
88  p_palier = np.array([ 236.70765333,  318.84163954,  347.89479113,  498.81428133,
89                        498.75945892,  616.52973518,  684.03994808,  715.74175462,
90                        773.50657693,  813.06343547,  869.73729183,  950.47108673,
91                        1063.71933298, 1104.99149298, 1207.06160606, 1251.80590983,
92                        1299.83040232, 1450.65749545, 1460.7952464 , 1580.89082548,
93                        1648.51071048, 1654.44222026, 1678.46438526, 1671.69636861,
94                        1684.11113599, 1674.9694633 , 1678.28689336, 1670.80268597,
95                        1670.59953769])
96
97  i_fluidisation = 20 #indice de la valeur de fluidisation
98  V_fixe = V[:i_fluidisation]
99
100  %% tracé de la courbe
101  a,b,c = np.polyfit(V_fixe, p_palier[:i_fluidisation], 2) #vérification d'une relation quadratique entre p et V
102  plt.figure()
103  plt.plot(V, p_f_3, 'o', color = 'royalblue')
104  p_palier_th = a*V_fixe**2 + b*V_fixe + c
105
106  #courbe modèle (ordre 2)
107  plt.plot(V_fixe, p_palier_th, '-', color = 'orangered')
108  plt.xlabel('V (vitesse du fluide en m.s-1)', fontsize = 15)
109  plt.ylabel('p (surpression en Pa)', fontsize = 15)
110  plt.title('modèle de Ergün', fontsize = 15)
111  plt.show()
112
113  #résidus
114  plt.figure()
115  plt.plot(V_fixe, p_palier[:i_fluidisation] - p_palier_th, 'o', color = 'royalblue')
116  plt.xlabel('V (vitesse du fluide en m.s-1)', fontsize = 15)
117  plt.ylabel('résidus (Pa)', fontsize = 15)
118  plt.title('tracé des résidus', fontsize = 15)
119  plt.show()
```

Annexe

1 - Tracé de courbes : relation surpression = $f(V_{\text{fluide}})$

```
121  ### tracé de la courbe
122  a,b = np.polyfit(V_fixe, p_palier[:i_fluidisation], 1) #vérification d'une relation linéaire entre p et V
123  plt.figure()
124  plt.plot(V, p_palier, 'o', color = 'royalblue')
125  p_palier_th = a*V_fixe + b
126
127  #tracé de la droite modèle (ordre 1)
128  plt.plot(V_fixe, p_palier_th, '-', color = 'orangered')
129  plt.xlabel('V (vitesse du fluide en m.s-1)', fontsize = 15)
130  plt.ylabel('p (surpression en Pa)', fontsize = 15)
131  plt.title('modèle de Darcy', fontsize = 15)
132  plt.show()
133
134  #résidus
135  plt.figure()
136  plt.plot(V_fixe, p_palier[:i_fluidisation] - p_palier_th, 'o', color = 'royalblue')
137  plt.xlabel('V (vitesse du fluide en m.s-1)', fontsize = 15)
138  plt.ylabel('résidus (Pa)', fontsize = 15)
139  plt.title('tracé des résidus', fontsize = 15)
140  plt.show()
141
142  ### détermination de la surpression au palier après la fluidisation + vérification que delta_p = m*g/S avec m = 400g
143  m,g = 400e-3,9.81
144  p_palier_exp = np.mean(p_palier[i_fluidisation:])
145  ecart = np.std(p_f_3[i_fluidisation:])
146  p_palier_theo = (m*g/S)
147  print('surpression constante après la fluidisation, palier théorique (Pa):',p_palier_theo, '. palier experimental (Pa):',p_palier_exp,
148        '. ecart normalisé: ',abs(p_palier_theo - p_palier_exp)/ecart)
```

Annexe

2 - Modélisation : création de la figure et initialisation des variables

```
1  ### importation des modules
2  import random as rd
3  import numpy as np
4  import matplotlib.pyplot as plt
5  import matplotlib.animation as animation
6
7  ### variables concernant le grain
8  rp = 2e-3 #rayon des grains (m)
9  m = 9e-6 #masse d'un grain (kg)
10 N = 300 #nombre de grains
11
12 ### création de la figure
13 fig, ax = plt.subplots()
14 x_lim,y_lim = (6e-2,1.4e-1)
15 fig.set_size_inches(5,10)
16 ax.set_xlim(-rp, x_lim + rp)
17 ax.set_ylim(-rp, y_lim + rp)
18 plt.xlabel("abscisse x et ordonnée y en mètres")
19 plt.title("simulation de "+str(N)+" grains")
20
21 for i in range(int(x_lim/(2*rp)) + 1):
22     plt.plot([i*2*rp,i*2*rp],[0,y_lim], "--", color = "#8099b2")
23 for i in range(int(y_lim/(2*rp)) + 1):
24     plt.plot([0,x_lim],[i*2*rp,i*2*rp], "--", color = "#8099b2")
25
26 ### variables temporelles
27 t = 0
28 dt = 3e-4 #faible devant T_colision
29
30 ### expressions déduites du PFD pour qu'il y ait peu d'oscillations:
31 Q = 0.65 #faibles oscillations
32 T_colision = 6e-3 #(s-1) ordre de grandeur de la durée de colision
33 pulsation = np.pi/T_colision #car le temps de colision correspond à une demi période du mouvement harmonique
34 g = 9.81
35
36 #valeurs des coefficients de frottement gamma et de constante de raideur K des grains et de la paroi:
37 K = m*pulsation**2
38 gamma = m*pulsation/Q
```

Annexe

2 - Modélisation : création de la figure et initialisation des variables

```
40  %% loi d'Ergun et Karman-Kozeni:
41  V = 0 #vitesse du fluide (modifiable pendant la simulation dans la console)
42  A = 150 #coefficient utilisé dans le calcul de la perméabilité
43  eta = 1.8e-5 #coefficient de viscosité dynamique de l'air (kg.m-1.s-1)
44  rho_air = 1.2e-3 #masse volumique de l'air (kg.m-3)
45  phi = np.zeros(N)
46
47  %% position, vitesse, acceleration:
48  X = np.array([rd.uniform(5*rp, x_lim - 5*rp) for k in range(N)])
49  Y = np.array([rd.uniform(5*rp, y_lim - 5*rp) for k in range(N)])
50  Vx, Vy = np.zeros(N), np.zeros(N)
51  Ax, Ay = np.zeros(N), np.zeros(N)
52
53  %% stockage des données des grains dans une liste Gn
54  Gn = []
55  for i in range(N):
56      grain = plt.Circle((X[i], Y[i]), rp, facecolor = rd.choice(["#62c1ff", "#2b8eff"]), edgecolor="#0082ca", fill=True)
57      ax.add_patch(grain)
58      Gn.append(grain)
```

Annexe

2 - Modélisation : répartition des grains et recherche des collision

```
60  ### detection des collisions
61  def repartition_tableau(X,Y):
62      n = len(X)
63      p,q = int(x_lim/(2*rp)) + 1, int(y_lim/(2*rp)) + 1
64
65      #tableau dont chaque case correspond à un carré de côté 2*rp
66      Lg = [[[ for j in range(q)] for i in range(p)]
67
68      #on place chaque grain dans sa case correspondante en fonction de ses coordonnées
69      for k in range(n):
70          i,j = int(X[k]/(2*rp)),int(Y[k]/(2*rp))
71          Lg[i][j].append(k)
72      return Lg
73
74  ### distance euclidienne au carré 0(1)
75  def dist_2(x1,x2,y1,y2):
76      return (x1 - x2)**2 + (y1 - y2)**2
77
78  ### étude des collisions voisines
79  def collisions_grain(Cn,Lg,X,Y,x,y,grain_i): #0(1) car on regarde seulement les grains voisins
80      lim_x,lim_y = len(Lg),len(Lg[0])
81      dp_carre = 4*rp**2
82
83      #pour un grain i donnée: étude des cases voisines qui sont dans le cadre de la figure
84      for p in (-1,0,1):
85          for q in (-1,0,1):
86              if x + p >= 0 and x + p < lim_x and y + q >= 0 and y + q < lim_y:
87
88                  #étude de chaque grain contenu dans la case
89                  for grain_j in Lg[x+p][y+q]:
90                      if grain_i != grain_j and grain_j not in Cn[grain_i] and dist_2(X[grain_i],X[grain_j],Y[grain_i],Y[grain_j]) <= dp_carre:
91                          #si il y a contact, on ajoute la collision entre i et j dans le dictionnaire
92                          Cn[grain_i].append(grain_j)
93                          Cn[grain_j].append(grain_i)
```

Annexe

2 - Modélisation : liste des collisions et calcul des forces de collisions entre les grains i et j

```

96  %% création de la liste des collisions
97  def collisions(X,Y,rp): #O(N)
98      #on trie chaque grain dans la case qui lui correspond
99      Lg = repartition_tableau(X,Y) #O(N)
100
101      #chaque grain peut entrer en collision avec les grains des cases voisines
102      N = len(X)
103      Cn = [[] for i in range(N)] #liste des collisions
104
105      #boucle en O(N)
106      for x in range(len(Lg)):
107          for y in range(len(Lg[x])):
108              for grain_i in Lg[x][y]:
109                  #on étudie les collisions pour chaque grain
110                  collisions_grain(Cn,Lg,X,Y,x,y,grain_i) #O(1)
111      return Cn,Lg
112
113  %% calcul des forces d'interaction
114  def force_grains(i,j,Vx,Vy,X,Y): #ressort + frottement normal et tangentiel
115
116      dx, dy = (X[j] - X[i]), (Y[j] - Y[i])
117
118      # Angle entre les grains
119      alpha = np.arctan2(dy, dx)
120      v_rel_x = Vx[i] - Vx[j]
121      v_rel_y = Vy[i] - Vy[j]
122
123      # Projection de la vitesse relative dans la direction normale et tangentielle
124      v_n = v_rel_x*np.cos(alpha) + v_rel_y*np.sin(alpha)
125      v_t = -v_rel_x*np.sin(alpha) + v_rel_y*np.cos(alpha)
126
127      d = np.sqrt(dx**2 + dy**2)
128
129      Fn = -K*(2*rp - d) - gamma*v_n
130      Ft = -gamma*v_t
131
132      return Fn, Ft, alpha
```

Annexe

2 - Modélisation : calcul du gradient de pression pour chaque grain

```
134  %% calcul de la fraction volumique pour un grain i donné
135  def maj_phi(Lg,X,Y,i):
136      #emplacement du grain dans le tableau Lg
137      x,y = int(X[i]/(2*rp)),int(Y[i]/(2*rp))
138      lim_x,lim_y = len(Lg),len(Lg[0])
139      N_grains_voisins = 0
140      N_cases = 0
141
142      #pour chaque grain, détermination du nombre de grains et de cases voisines
143      for p in (-1,0,1):
144          for q in (-1,0,1):
145              if x + p >= 0 and x + p < lim_x and y + q >= 0 and y + q < lim_y:
146                  N_grains_voisins += len(Lg[x+p][y+q])
147                  N_cases += 1
148
149      #on détermine le volume occupé uniquement par les grains
150      Vg = N_grains_voisins*(4/3)*np.pi*rp**3
151
152      #on détermine le volume occupé par les grains et l'air
153      V0 = N_cases*(2*rp)**3
154
155      return Vg/V0
156
157  %% calcul de la permeabilité k avec la relation de Karman-Kozeni
158  def maj_k(phi_grain):
159      return ((2*rp)**2)*((1-phi_grain)**3)/(A*(phi_grain**2))
160
161  %% mise a jour du gradient de pression
162  def gradient_pression(i,Lg,X,Y):
163      phi[i] = maj_phi(Lg,X,Y,i)
164      k = maj_k(phi[i])
165      #loi d'Ergun
166      return rho_air*g + (eta/k)*V + 1.8*((rho_air*phi[i])/(2*rp*(1 - phi[i])**3))*V**2
```

Annexe

2 - Modélisation : résultante des forces et conditions aux limites

```
168  #%% conditions aux limites
169  def conditions_limites(x,y,vxi,vyi,fxi,fyi): #pour éviter que des grains sortent du cadre
170      #forces de contact avec la paroi
171      if x > x_lim - rp:
172          fxi = -abs(K*(x_lim - rp - x)) - gamma*vxi
173      elif x < rp:
174          fxi = abs(K*(rp - x)) - gamma*vxi
175      if y < rp:
176          fyi = abs(K*(y - rp)) - gamma*vyi
177      elif y > y_lim - rp:
178          fyi = -abs(K*(y_lim - rp - y)) - gamma*vyi
179
180      #inversion de la composante normale de vitesse a proximité de la paroi
181      if y > y_lim - rp/2 or y < rp/2:
182          vyi = - vyi
183      if x > x_lim - rp/2 or x < rp/2:
184          vxi = - vxi
185      return vxi,vyi,fxi,fyi
186
187  #%% resultante des forces
188  def forces(X,Y,Vx,Vy):
189      #on détermine les colisions qui existent entre les grains
190      Cn,Lg = collisions(X,Y,rp)
191
192      #initialisation des forces
193      Fx,Fy = np.zeros(N), np.zeros(N)
194      Fy += - m*g
195
196      #on ajoute les forces entre chaque grain qui sont en colision
197      for i in range(N):
198          for j in Cn[i]:
199              Fn,Ft,alpha = force_grains(i,j,Vx,Vy,X,Y)
200              Fx[i] += Fn*np.cos(alpha) - Ft*np.sin(alpha)
201              Fy[i] += Fn*np.sin(alpha) + Ft*np.cos(alpha)
202
203      #on rajoute la force exercée par le fluide dans le milieu poreux sur chaque grain
204      Pf = gradient_pression(i,Lg,X,Y)
205      Fy[i] += Pf*(4/3)*np.pi*rp**3
206
207      #conditions aux limites: parois laterales et horizontales
208      Vx[i],Vy[i],Fx[i],Fy[i] = conditions_limites(X[i],Y[i],Vx[i],Vy[i],Fx[i],Fy[i])
209
210      return Fx, Fy
```

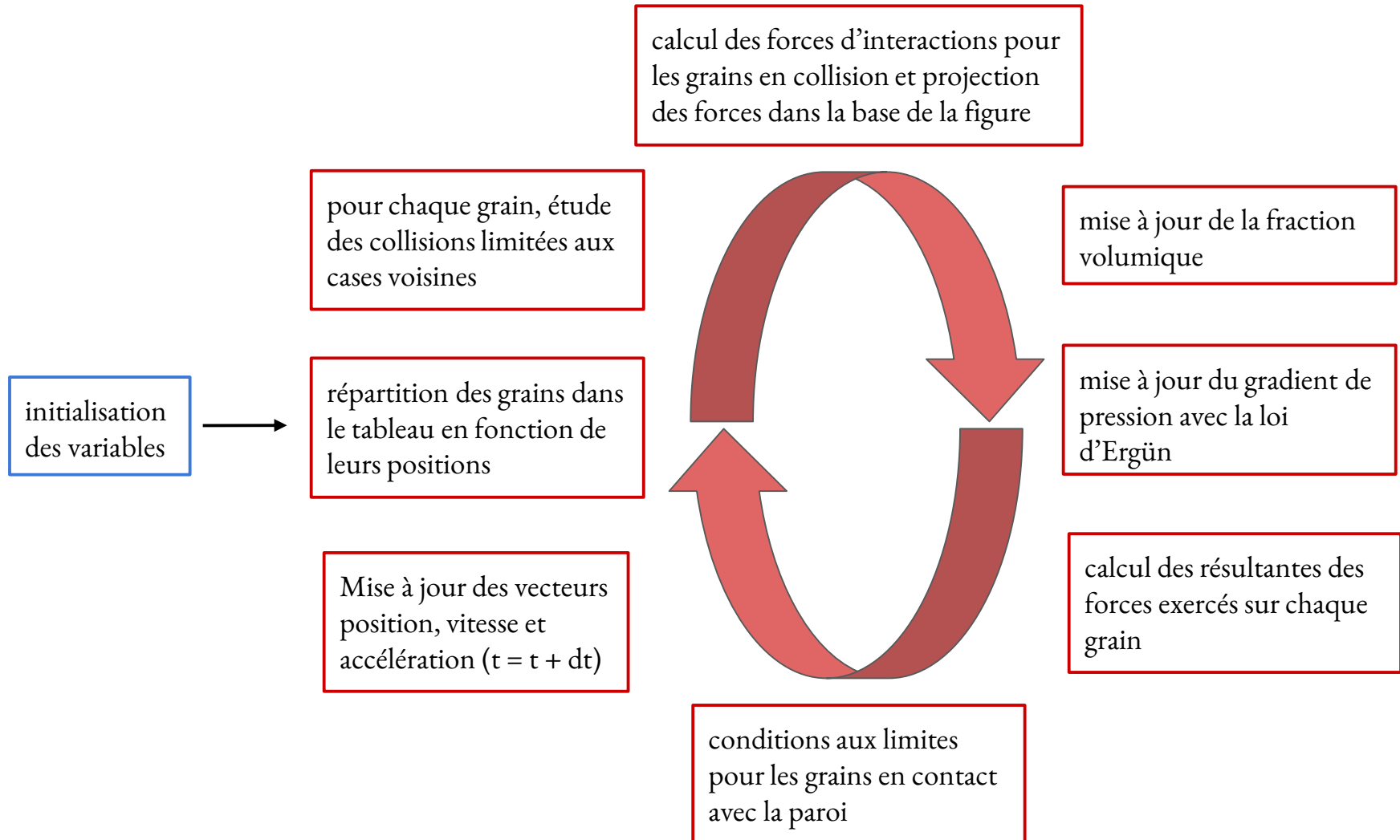
Annexe

2 - Modélisation : intégration Verlet et mise à jour des positions

```
206 - ### intégration par méthode de Verlet (Euler est instable)
207 - def integration(X,Y,Vx,Vy,Ax,Ay):
208
209     #nouvelles listes d'accélération, de vitesse, de position
210     Ax2, Ay2 = np.zeros(N), np.zeros(N)
211     Vx2, Vy2 = np.zeros(N), np.zeros(N)
212     X2, Y2 = np.zeros(N), np.zeros(N)
213
214     #intégration Verlet
215     Vx_demi, Vy_demi = Vx + 0.5*dt*Ax, Vy + 0.5*dt*Ay
216     X2, Y2 = X + dt*Vx_demi, Y + dt*Vy_demi
217     Fx, Fy = forces(X2, Y2, Vx_demi, Vy_demi)
218     Ax2, Ay2 = Fx/m, Fy/m
219     Vx2, Vy2 = Vx_demi + 0.5*dt*Ax2, Vy_demi + 0.5*dt*Ay2
220
221     return Ax2,Ay2,Vx2,Vy2,X2,Y2
222
223 - ### mise à jour des positions
224 - def maj(frame):
225     global Ax,Ay,Vx,Vy,X,Y,rp,t
226
227     #nouvelles positions au temps t + dt
228     Ax,Ay,Vx,Vy,X,Y = integration(X,Y,Vx,Vy,Ax,Ay)
229
230     for i,grain in enumerate(Gn):
231         grain.set_center((X[i], Y[i]))
232     t += dt
233
234     return Gn
235
236 - ### début de l'animation
237 ani = animation.FuncAnimation(fig, maj, frames=1, interval=dt*1e3, blit=True)
238 plt.show()
```

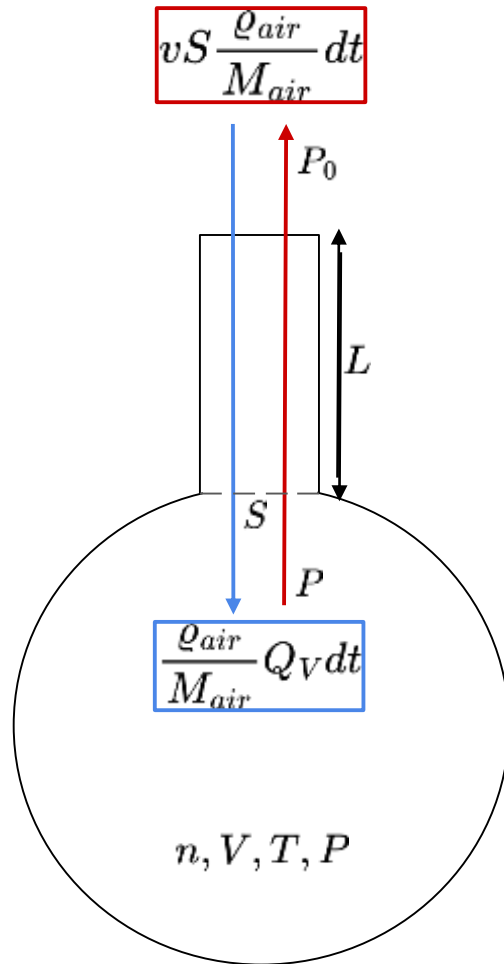
Annexe

schéma détaillé du fonctionnement du modèle numérique



Annexe

démonstration de l'équation de la pression dans la cocotte minute



Bilan de matière dans la cocotte minute :

$$d(n) = \delta n_{echange}$$

$$donc \frac{V}{RT} dP = \frac{\rho_{air}}{M_{air}} Q_V dt - vS \frac{\rho_{air}}{M_{air}} dt$$

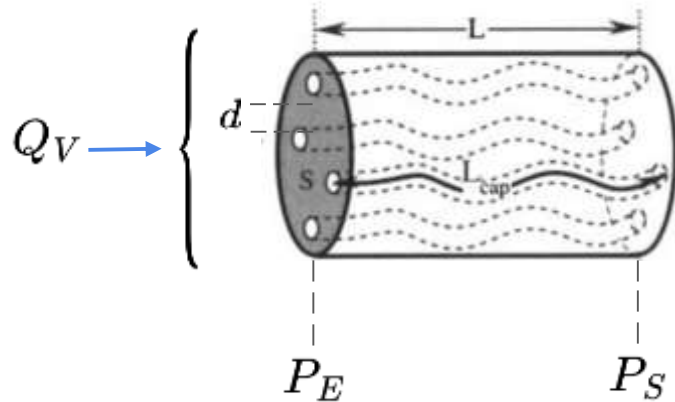
DL₀ de v au voisinage de $t = 0$:

$$v = v_0 = 0$$

$$donc en integrant : P(t) = P_0 + Q_V \frac{\rho_{air}}{M_{air}} \frac{RT}{V} t$$

Annexe

démonstration de la loi de Darcy



En moyennant les débits volumiques des N pores :

$$Q_V = N \langle Q_{V_{un\ pore}} \rangle = N \frac{\pi d^4}{128\eta} \frac{\Delta P}{L_{cap}}$$

La vitesse moyenne vaut donc :

$$V = \frac{Q_V}{S} = \frac{k}{\eta} \frac{\Delta P}{L}$$

en posant $k = N \frac{\pi d^4}{128S} \frac{L}{L_{cap}}$

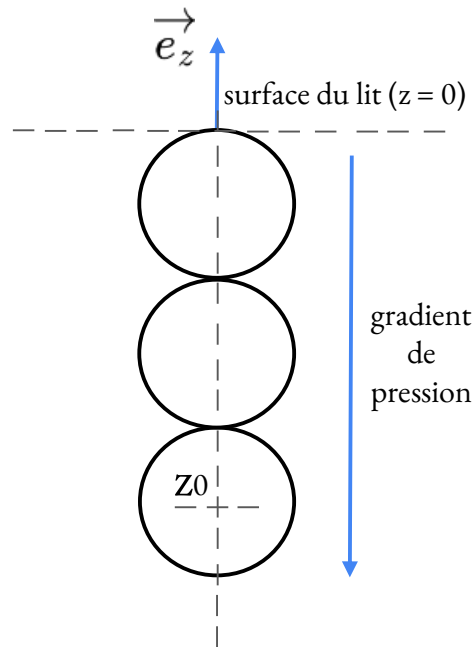
En appliquant la loi de Poiseuille
pour chaque pore :

$$v(r) = \frac{-\Delta P}{4\eta L_{cap}} \left(r^2 - \frac{d^2}{4} \right)$$

$$\text{donc : } Q_{V_{un\ pore}} = \int_0^{d/2} v(r) dS = \frac{\pi}{128\eta} \frac{\Delta P}{L_{cap}} d^4$$

Annexe

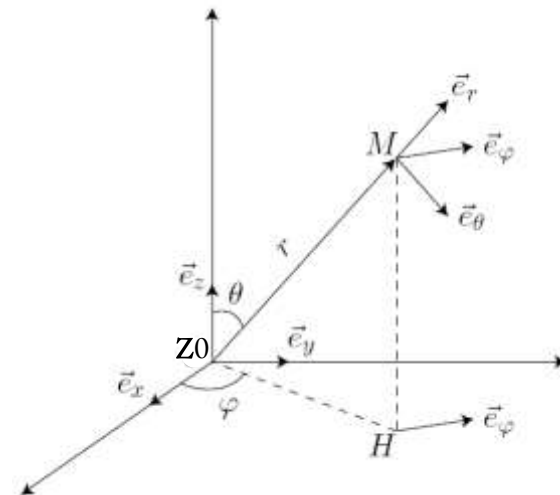
démonstration : force de pression fluide appliqué sur un grain



Soit $\frac{\partial P}{\partial z}(V, \phi)$ le gradient de pression
 défini par la loi d'Ergün :
 on a $P_{\text{fluide}} = -\frac{\partial P}{\partial z} z$

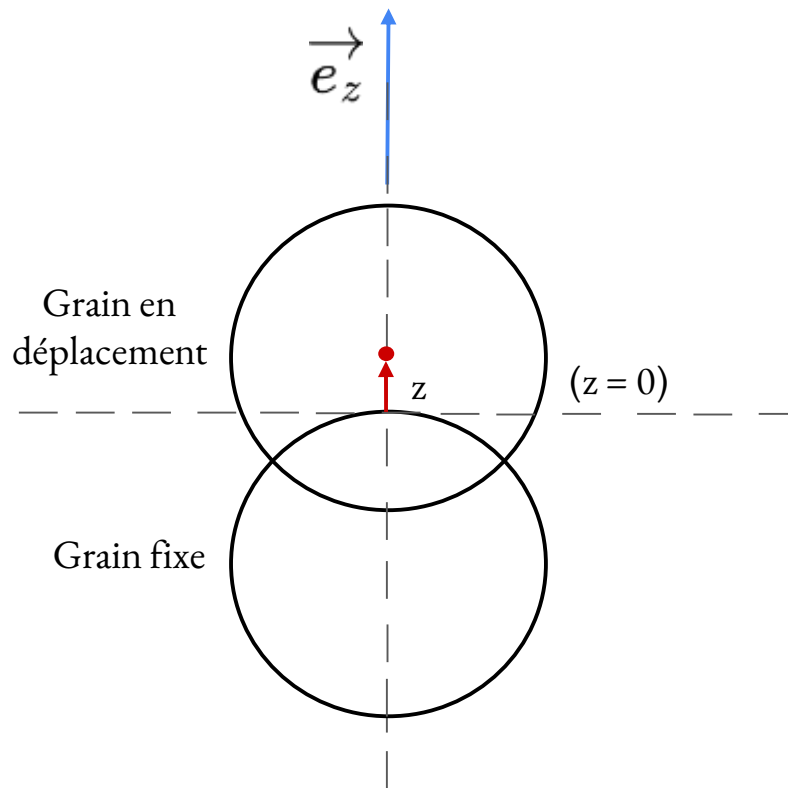
par symétrie :

$$\begin{aligned}\vec{F}_p &= \left(\oint_{\text{Grain}} P_{\text{fluide}} d\vec{S} \cdot \vec{e}_z \right) \vec{e}_z \\ \vec{F}_p &= \frac{\partial P}{\partial z} \int_0^{2\pi} d\varphi \int_0^\pi (z_0 + r \cos(\theta)) r^2 \cos(\theta) \sin(\theta) d\theta \vec{e}_z \\ \vec{F}_p &= 2\pi r^2 \frac{\partial P}{\partial z} \left[z_0 \frac{\cos^2(\theta)}{2} + r \frac{\cos^3(\theta)}{3} \right]_0^\pi \vec{e}_z \\ \vec{F}_p &= \frac{4\pi}{3} r^3 \frac{\partial P}{\partial z} \vec{e}_z\end{aligned}$$



Annexe

démonstration : détermination des constantes de frottement et de raideur



Principe fondamental de la dynamique

$$(Oz) : \frac{d^2 z}{dt^2} + \frac{\gamma}{m} \frac{dz}{dt} + \frac{k_N}{m} (z - r) = -g$$

$$\text{peu d'oscillations : } \frac{\sqrt{mk_N}}{\gamma} = Q \simeq 0.5$$

$$\text{et } w_0 = \frac{2\pi}{2T_{\text{colision}}} \text{ avec } T_{\text{colision}} \simeq 6.10^{-3} \text{ s}$$

$$\text{or } w_0^2 = \frac{k_N}{m}$$

d'ou un ordre de grandeur des constantes :

$$k_N = mw_0^2 \text{ et } \gamma = \frac{mw_0}{Q}$$

Annexe

Détermination des caractéristiques des différents milieux granulaires



Micromètre pour la mesure du rayon



	Sable fin	Lentilles	Sel	Graines de chia	Quinoa
Rayon moyen (mm)	0,50	1,75	1,50	0,65	0,90
Masse volumique (kg.m ⁻³)	1850	1547	848	853	1457
Vitesse de fluidisation (m.s ⁻¹)	1,08	0,80	0,85	0,64	0,63